

AgentOS-uCore 设计开发文档



参赛团队 project61-agentOS-happylegend
参赛学校 电子科技大学
团队成员 王浩沣、康峻豪
项目导师 余盛季、郭力维

2026 年

摘要

我们关注的是大模型驱动的智能体工作流在教学操作系统中的系统支持问题。当前大量智能体平台运行在普通用户态之上，具有部署灵活、策略更新方便的优势；但智能体的工具调用、上下文、文件访问、事件等待和多智能体协作往往分散在日志、状态文件和用户态约定中。对于需要长期运行、失败恢复和结果复查的任务，这种方式会带来状态可信性不足、文件扫描成本高、轮询等待浪费、权限检查分散和来源关系难以追踪等问题。普通内核只能看到进程读写文件和发起系统调用，难以理解“某个工具调用为何发生、结果来自哪些输入、等待中的智能体应由哪个事件唤醒”等智能体工作流事实。

针对这些问题，我们以 uCore 教学操作系统为基础，设计并实现 AgentOS-uCore 通用内核支持层。我们从三个核心层面扩展原有内核机制：首先，在进程和地址空间层面引入智能体身份、能力位、上下文区和资源状态，使普通进程与智能体进程能够共存，并保证智能体身份由内核确认；其次，在工具调用和文件对象层面提供结构化请求、批量执行、参数校验、结果记录、文件对象元数据、内容摘要和租约检查，使智能体能够以可解释方式访问系统对象；最后，在运行循环和可观测层面实现事件队列、订阅等待、心跳、审计记录、运行时间线、来源追踪和大模型请求记录，使长期运行的智能体能够通过内核等待、唤醒、记录和复查。

同时，我们构建了配套的用户态科研智能体平台、普通 uCore 对照目标、宿主机结果提取脚本和多组实验图表。科研平台作为演示负载，覆盖资料读取、方法检查、分析复核、报告撰写、大模型转发和失败恢复；内核专项测试覆盖智能体创建、上下文区、工具调用、文件对象查询、事件等待、权限控制、调度提示和大模型模板转发；双目标对照在相同输入下比较普通用户态路径与 AgentOS-uCore 路径的扫描量、上下文重建成本、轮询次数和并发写入风险。我们把测试结果以 QEMU 输出、状态文件、原始 CSV、统计表和图表共同保存，用于验证 AgentOS-uCore 在真实多智能体流程中的功能完整性和系统行为变化。

目录

1	概述	4
1.1	项目背景及意义	4
1.2	现有研究调研	6
1.3	系统整体架构设计	8
1.4	项目创新点	10
1.5	项目目标完成情况	11
1.6	项目开发情况	12
1.7	与操作系统内核机制的对应关系	12
1.8	团队成员分工	14

2	模块设计和实现	15
2.1	智能体进程层设计	16
2.1.1	进程状态和生命周期	17
2.1.2	地址空间处理	17
2.1.3	异常处理和测试证据	18
2.1.4	与 uCore 原有机制的关系	18
2.2	结构化工具调用运行时	19
2.2.1	调用路径	20
2.2.2	参数和返回码	21
2.2.3	权限语义	22
2.3	上下文路径与可信历史	22
2.3.1	写入规则	23
2.3.2	查询和淘汰	24
2.4	文件对象元数据与内容摘要查询	24
2.4.1	元数据模型	25
2.4.2	查询流程	25
2.4.3	租约和冲突处理	26
2.5	事件驱动的智能体运行循环	26
2.5.1	事件类型和队列行为	27
2.5.2	与普通轮询的差异	28
2.6	审计记录、运行时间线、来源追踪与大模型友好机制	28
2.6.1	记录组织方式	29
2.6.2	大模型请求处理	29
2.6.3	测试证据	29
2.7	关键设计取舍	30
2.8	科研智能体平台演示负载	31
3	项目测试结果	32
3.1	测试体系设计	32
3.2	AgentOS 专项测试	34
3.3	测试命令与验收标记	35
3.4	工具调用吞吐实验	36

3.5	文件对象查询实验	36
3.6	上下文与运行时间线查询实验	37
3.7	事件等待与并发写入实验	38
3.8	LLM Relay 模式实验	40
3.9	恢复流程成本实验	41
3.10	双目标科研平台对照	42
3.11	实验结果解读方式	46
4	总结与展望	47
4.1	工作总结	47
4.2	项目收获	49
4.3	系统适配能力	49
	参考文献	49
	AI 辅助工具使用说明	50
	开源项目声明	51

1 概述

1.1 项目背景及意义

近年来，大模型应用正在从单次问答走向能够连续执行任务的智能体系统。以 Claude Code 一类工具为例，一个典型任务通常包含环境观察、计划生成、工具选择、命令执行、结果读取和下一步决策等多个环节 [2]。ReAct 工作将推理轨迹与外部行动交替组织，说明模型在行动后获得的新信息会影响后续计划 [3]。SWE-agent 的研究进一步指出，语言模型智能体已经成为一种新的计算机使用者，面向智能体设计的计算机接口会直接影响其编辑代码、浏览仓库和运行测试的效果 [4]。

面向操作系统观察，一个智能体工作流由三个关键部分构成：第一，智能体进程负责规划任务、调用工具、接收反馈和继续行动；第二，内核支持层负责维护身份、上下文、文件对象元数据、事件队列、权限能力和运行记录；第三，用户态策略层负责科研流程、代码分析、系统管理、大模型转发和结果导出等具体业务。三者之间通过系统调用、智能体上下文区、事件通知和状态文件完成双向交互：用户态向内核提交结构化行动请求，内核记录请求、检查权限、更新对象状态并把结果返回给智能体，后续智能体再根据这些结果决定下一步。

这种架构保留了用户态策略的灵活性，也暴露出普通操作系统对智能体工作流支持不足的问题。具体而言，普通进程只能表示可执行文件的一次运行实例，难以区分计算任务、工具执行任务和长期运行的智能体任务；普通系统调用只携带原始参数，无法直接表达工具名称、参数键值、返回结构和错误语义；普通文件系统主要按路径读写文件，难以回答“查询某类对象”“读取内容摘要”“寻找与上一步结果相关的文件”等语义化请求；普通日志便于人工排查，却难以形成可查询、可回放、可验证的上下文路径。

上述问题在一次典型工具调用中会集中出现。普通用户态路径往往需要先从多个文件恢复当前任务状态，再由应用自行解析工具名称和参数，执行后把结果写回日志或状态文件，最后由其他智能体轮询这些文件判断是否继续执行。主要开销来自重复读取、重复解析、结果可信性不足和等待方式不明确。AgentOS-uCore 将这条路径拆成内核能够管理的若干步骤：调用者身份由进程状态确认，参数由结构化工具协议校验，结果写入上下文路径和审计记录，相关文件对象通过元数据查询定位，等待者通过事件队列唤醒。这样，同一次行动在执行、恢复和复查时都能找到稳定的系统证据。

以科研智能体工作流为例，用户通常希望系统完成资料读取、方法检查、实验分析、复核、报告撰写和失败恢复。若完全停留在普通用户态实现，平台需要自行维护一组状态文件：某个阶段是否完成、哪个智能体正在处理、上一次大模型请求产生了什么结论、失败阶段是否已经重试、最终报告引用了哪些输入。状态文件越多，

恢复时越依赖约定命名和日志拼接；多个智能体同时写入同一结果时，还需要额外锁文件或短期协调字段避免互相覆盖。此类问题并不只属于科研场景，代码智能体、系统管理员智能体、数据流水线智能体和写作智能体都会遇到类似情况：它们都需要把外部行动、文件对象、上下文、事件和结果联系起来。

普通操作系统也能运行这些应用，但它看到的往往只是进程读写文件、执行系统调用、等待时间片和退出状态。智能体真正关心的“这次工具调用为什么发生”“这个报告段落来自哪个文件和哪次大模型响应”“恢复动作是否已经提交过”“等待中的智能体是否应被唤醒”，在普通内核视角下缺少结构化表达。

赛题提出的 AgentOS 方向为这类问题提供了操作系统侧的切入点 [2]。一方面，智能体身份进入进程模型后，内核可以区分普通程序和长期运行的智能体，并按能力位管理工具、文件和事件访问；另一方面，智能体上下文区把多轮行动记录放到内核可维护的位置，既保留用户态直接读取的性能路径，也提供可信快照；同时，文件对象元数据、事件队列、等待唤醒和审计记录使内核能够保存“行动、对象、结果、原因”之间的关系。由此，科研流程、提示词、大模型转发和结果导出仍由用户态决定，内核负责保存通用事实和执行必要检查。

我们以 uCore 教学操作系统为基础设计 AgentOS-uCore，目标是在教学内核中提供一组面向 AI Agent / LLM 工作流的通用支持机制。综合上述背景分析，我们将主要目标确定为：

1. 构建智能体进程管理机制：在进程模型中加入智能体身份、角色、能力位、资源配额、心跳、事件队列和可观测状态，使普通进程与智能体进程能够共存，并保证普通进程不能伪造智能体身份。
2. 构建可信上下文与结构化工具调用机制：为每个智能体提供内核权威副本和用户态镜像组成的上下文区，支持按名称调用和批量调用两种工具路径，由内核完成参数校验、权限检查、结果记录和上下文追加。
3. 构建面向智能体的文件对象查询机制：把文件元数据、内容摘要和真实文件对象关联起来，使智能体能够按属性、状态、类型、摘要和依赖关系查询文件，减少普通路径扫描带来的开销。
4. 构建事件驱动的智能体运行机制：支持事件订阅、睡眠等待、超时、心跳、取消等待和多智能体消息唤醒，使长期运行的智能体在无事件时让出 CPU，并在需要处理任务时及时恢复运行。
5. 构建审计记录、运行时间线和来源追踪机制：把工具调用、事件、文件对象、大模型请求和产物之间的关系组织为可查询证据，便于问题定位、验证、结果复查和异常恢复。

我们让科研智能体平台承担演示负载、压力负载和对照负载的作用。它通过通用 AgentOS 机制完成科研流程编排、大模型转发、报告生成和结果导出，用于呈现

内核机制在真实多智能体任务中的运行效果。同一套机制也可服务代码智能体、系统管理员智能体、数据流水线智能体、写作智能体和游戏 NPC 智能体等不同场景。

AgentOS-uCore 的实现落在 uCore 的内核路径中，而不是只在用户态增加一层应用程序。后续章节围绕进程创建、地址空间、系统调用、文件系统、时钟中断、等待唤醒、并发同步和 RISC-V/QEMU 运行环境展开。每一类机制都说明它在 uCore 原有结构中的位置、AgentOS-uCore 的扩展方式、用户态如何调用以及测试如何验证。

1.2 现有研究调研

围绕智能体系统的研究可以从两条线索理解。第一条线索来自大模型应用本身，重点解决模型如何观察环境、调用工具、保存记忆、组织多智能体协作。第二条线索来自操作系统和系统软件，重点解决内核如何提供统一接口、低开销观测、可信记录和资源管理。前者已经形成较成熟的用户态生态，后者为 AgentOS-uCore 的内核设计提供了参照。

用户态智能体框架的优势在于灵活。开发者可以快速接入云端大模型、提示词模板、数据库、网页工具和任务编排逻辑，也可以按应用需求设计不同角色和工作流。然而，灵活性通常依赖日志、状态文件、数据库字段和应用约定来维持。当任务跨越多轮行动、多个智能体和多个文件对象时，系统层仍然只能看到普通进程和普通文件访问，难以直接理解“工具为什么被调用、结果来自哪里、哪个等待者应该被唤醒、谁有权限更新某个对象”。

首先，在智能体推理与行动范式方面，ReAct 提出将自然语言推理和外部行动结合起来，让模型在每一步行动后读取新结果，并据此调整后续推理过程 [3]。这种方法使大模型能够处理搜索、问答和交互式任务，其核心是持续观察环境、调用工具、读取反馈并更新状态。SWE-agent 面向软件工程任务提出 Agent-Computer Interface 的概念，把语言模型智能体看作新的计算机用户，通过更适合智能体的命令、反馈和环境表示提升任务完成能力 [4]。这类工作说明了智能体的运行形态，也说明操作系统接口会影响智能体能否稳定完成真实任务。

其次，在工具调用和上下文标准化方面，Anthropic 的工具调用文档给出了清晰的工程流程：模型根据用户请求和工具描述生成结构化工具调用，应用或服务端执行工具，再把工具结果返回给模型 [6]。该流程已经把工具名称、参数、执行结果和错误处理作为智能体运行的基本组成部分。MCP 进一步把数据源、工具和工作流抽象为 AI 应用可以连接的外部能力，使不同应用能够用统一方式访问文件、数据库、服务和外部系统 [7]。这些协议提高了应用之间的互操作能力，但工具调用记录、权限检查、上下文保存和事件唤醒仍主要由各个应用自行维护。

再次，在多智能体协作和长期工作流方面，AutoGen 支持多个智能体围绕任务

进行对话、分工和工具使用 [8]。LangGraph 强调长期运行的有状态智能体，提供持久执行、人工介入、记忆和执行轨迹可视化等能力 [9]。这些框架表明，多智能体应用需要保存状态、恢复任务、观察执行过程并协调不同角色。它们擅长编排应用策略、接入真实模型和组织复杂任务，但关键状态大多位于数据库、日志或普通文件中，操作系统难以为上下文可信、权限控制、事件等待和来源追踪提供统一支撑。

最后，操作系统领域已有若干工作为 AgentOS-uCore 提供了类比参考。FUSE 通过内核模块、用户态守护进程和通信接口构成用户态文件系统框架，体现了复杂策略放在用户态、通用转发机制放在内核中的设计思路 [10]。FUSE 的性能问题也说明，频繁穿越用户态和内核态会带来上下文切换、数据拷贝和队列等待开销。eBPF 提供了在内核中安全执行小程序、低开销观测系统行为和扩展内核能力的路径 [11]。Linux 审计系统说明，操作系统可以记录安全相关事件，用于追踪用户、进程和系统调用活动 [12]。W3C PROV 给出了来源信息模型，用实体、活动和人员之间的关系描述数据或结果如何产生 [13]。这些工作共同表明，内核适合保存高可信、低歧义、便于复查的运行事实。

综合上述研究，我们认为现有用户态智能体框架已经较好地解决了大模型调用、工具编排、多智能体协作和工作流持久化等问题，工具协议和来源追踪模型也逐渐成形。操作系统层面仍缺少面向智能体的统一支撑：进程模型缺少智能体身份，系统调用缺少工具调用语义，文件访问缺少面向智能体的属性和摘要查询，用户态日志难以形成可信上下文，长期任务等待缺少统一睡眠和唤醒机制，多智能体协作缺少统一能力检查与审计记录。我们围绕这些问题开展 AgentOS-uCore 设计，在教学内核中实现智能体进程、上下文区、结构化工具调用、文件对象查询、事件运行、运行时间线、审计记录和来源追踪，并通过科研智能体平台验证其在真实负载中的作用。

表 1 总结了现有工作与我们关注问题之间的关系。可以看到，已有框架更擅长表达智能体策略和应用编排，操作系统侧已有工作更擅长处理可信记录、观测和资源管理。我们把二者之间的缺口作为切入点，在 uCore 内核中实现一组通用机制，让智能体应用能够以结构化方式使用文件、上下文、事件和来源证据。

表 1: 现有工作与 AgentOS-uCore 的关系

类别	已有能力	我们补充的系统能力
推理与行动范式	ReAct、SWE-agent 说明了观察、行动、反馈、再决策的任务形态。	内核记录行动原因、工具结果、事件唤醒和执行顺序，形成可查询路径。
工具协议与上下文连接	工具调用文档和 MCP 使用户态应用能够连接外部工具和数据源。	系统调用层提供工具表、参数校验、错误码、批量执行和权限检查。

类别	已有能力	我们补充的系统能力
多智能体工作流	AutoGen、LangGraph 支持多角色协作、记忆和任务编排。	进程层提供智能体身份、能力位、消息、事件等待、心跳和审计记录。
操作系统观测与追踪	FUSE、eBPF、Linux Audit、W3C PROV 提供了用户态策略、内核观测和来源描述方法。	文件对象查询、运行时间线、审计记录和来源追踪统一进入 AgentOS 机制。

从写作和设计逻辑上看，上述工作给了我们三点启发。第一，智能体应用层已经拥有成熟的编排方式，因此内核不需要接管提示词、任务分工或具体报告内容；第二，工具调用和上下文协议已经说明外部行动必须结构化，因此系统调用层需要提供稳定的参数、返回和错误语义；第三，操作系统已有的审计、观测和来源模型说明内核适合保存高可信、低歧义的运行事实。我们后续模块设计均围绕这三点展开：策略留给用户态，事实进入内核，二者通过结构化接口连接。

1.3 系统整体架构设计

综合上述背景分析和调研结果，我们认为智能体工作流的主要矛盾集中在两个方面：应用层需要保留足够灵活的策略，内核层需要保存和管理身份、上下文、文件对象、事件和运行记录等通用事实。若所有状态都停留在用户态，恢复、复查和协作会依赖大量约定；若把具体业务逻辑下沉到内核，系统又会失去通用性。因此，我们选择以 uCore 为基础，将智能体相关的通用机制放入内核，将科研流程、大模型转发和结果导出等策略留在用户态。

AgentOS-uCore 的整体结构如图 1 所示。我们把最上层设计为科研智能体平台、宿主机结果工具和大模型转发进程，也可以替换成代码智能体、运维智能体或数据流水线智能体。中间是用户态接口层，负责把应用请求转换为结构化系统调用。内核中依次提供智能体进程层、工具运行时、上下文路径层、文件对象查询层、事件运行层和可观测层。围绕这套结构，我们从三个核心路径支撑智能体负载：面向执行的进程与上下文路径，面向文件对象和事件的系统服务路径，面向复查和结果生成的证据路径。

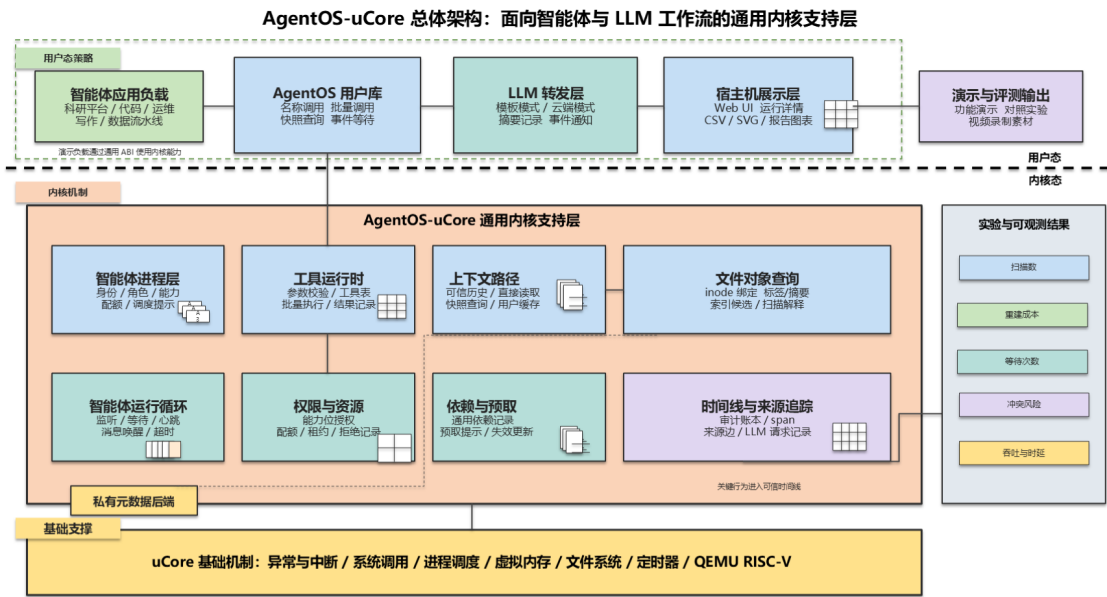


图 1: AgentOS-uCore 总体架构

仓库根目录是增强版 AgentOS-uCore。未修改 uCore 对照目标放在 baseline uCore 目录，用于运行同一科研平台负载并提供普通路径结果。内核源码、用户态程序、宿主机结果工具、脚本、文档和图表分别放在根目录下的对应目录中。这样的目录安排让增强内核、对照目标和测试产物保持清晰对应。

我们把项目运行链路分为三步。第一步，QEMU 分别运行未修改 uCore 与增强版 uCore，二者使用相同科研请求和相同输入数据。第二步，宿主机工具从文件系统镜像中提取状态文件、日志、CSV 和 JSON，检查两个目标是否完成同一组平台功能。第三步，汇总脚本生成实验图表和状态摘要，用于比较普通路径与 AgentOS 路径在扫描、上下文、事件、并发写入、来源追踪和大模型转发上的差异。

一、面向智能体执行的进程与上下文路径

我们让智能体执行路径从进程身份开始。用户态通过创建接口得到智能体进程，内核在进程控制块中记录身份、能力位、上下文页、事件队列和可观测统计。之后，智能体每次执行工具、等待事件或更新对象，都会把关键摘要写入内核权威历史。用户态可以直接读取镜像获得较低开销的观察结果，也可以通过快照接口读取可信记录。该路径对应赛题中的智能体进程、智能体上下文、工具调用和上下文路径要求。

二、面向文件对象和事件的系统服务路径

智能体工作流中大量操作都围绕文件对象展开，例如读取输入、查找证据、更新报告、处理失败阶段。我们将文件对象元数据与真实文件对象绑定，并通过索引

候选集减少普通路径的目录扫描。同时，文件变化、恢复动作、大模型响应和多智能体消息都能转化为事件，等待者在无事件时进入睡眠。该路径把文件查询、事件等待和多智能体协作连接起来，使长期任务可以依赖内核服务推进，而不需要反复读取约定格式的状态文件。

三、面向复查和结果说明的证据生成路径

内核运行事实需要保留稳定的提取路径。我们将工具调用、事件、审计、来源关系和调度原因写成可提取状态，宿主机脚本再把 QEMU 输出、状态文件、CSV 和 JSON 汇总为图表与摘要。该路径不改变内核机制本身，它的作用是把内核事实转化为可复查材料。双目标对照正是基于这条路径运行：普通目标提供用户态实现结果，增强目标提供同一任务下的内核证据和成本变化。

表 2: 系统分层与主要职责

层次	组成	主要职责
内核机制层	智能体进程、上下文区、工具运行时、文件对象查询、事件队列、审计记录。	提供通用内核机制，负责身份、权限、记录、查询、等待和可观测状态。
uCore 用户态层	专项测试程序、科研平台程序、大模型转发进程、普通对照程序。	调用通用 AgentOS 接口，实现具体任务策略和演示负载。
宿主机工具层	镜像提取、结果比较、图表生成和状态摘要。	从 QEMU 输出和镜像状态中生成可复查材料，服务测试和结果说明。
双目标对照层	未修改 uCore 目标与 AgentOS-uCore 目标。	在相同输入上比较普通用户态路径和内核增强路径的差异。

1.4 项目创新点

1. 智能体成为内核可识别对象。智能体身份进入进程控制块，内核直接维护上下文位置、能力位、心跳、事件队列和调度状态，普通进程的自报字段不会改变内核身份。
2. 上下文区同时支持可信记录和高速读取。内核权威副本负责可信历史，用户态镜像负责高频读取，用户自管缓存负责策略数据。可信查询和快速读取有明确的数据来源。
3. 工具调用具有结构化协议。工具拥有名称、编号、参数类型、返回结构和错误

码，支持按名称调用和批量执行。内核在执行前完成参数与权限检查，失败请求不会产生工具副作用。

- 4. **文件查询从路径搜索扩展为对象语义查询。**文件可以附加元数据和内容摘要，查询结果返回候选数量、触达记录数和查询计划，智能体能够理解查询成本和结果来源。
- 5. **长期运行智能体采用事件驱动。**智能体注册订阅条件后进入睡眠等待，由事件、消息、心跳或超时唤醒，避免用户态循环检查状态文件。
- 6. **运行事实可以被复核。**工具调用、事件、调度、预取和大模型请求进入审计记录、运行时间线和来源追踪，用户态平台可以从内核读取证据并生成可视化材料。
- 7. **大模型调用被纳入系统管理视野。**内核记录请求编号、摘要、目标进程、超时、状态和结果事件；真正的云端调用、密钥和网络连接由用户态或宿主机转发层处理。

1.5 项目目标完成情况

表 3: 赛题任务完成情况

目标	当前状态	说明
任务一: 智能体进程创建与地址空间设计	已实现	内核增加智能体身份、角色、能力位、上下文元信息、事件队列、调度记录和资源状态。智能体上下文区固定映射为 6 页，创建、执行新程序和退出路径都维护对应状态。
任务二: 结构化工具调用接口	已实现并增强	支持按工具名和参数键值调用，也支持按工具编号批量执行。内置工具覆盖进程查询、文件查询、消息、上下文、事件、通用动作、产物更新、大模型请求和依赖记录。
任务三: 上下文路径管理	已实现并增强	每个智能体维护 128 条上下文记录和细节记录，包含调用顺序、原因、跨度、时间、参数摘要、结果摘要和完整性摘要，支持查询、快照、清理和回滚。
任务四: 面向智能体查询优化的文件系统扩展	已实现	文件元数据绑定真实文件对象，私有后端保存 128 条记录。系统支持扫描路径、索引路径、内容摘要、查询缓存、预取提示和文件编辑租约。

目标	当前状态	说明
任务五: 智能体运行循环	已实现	支持事件队列、订阅、取消订阅、睡眠等待、超时、心跳停止、取消等待、多智能体消息唤醒和调度提示记录。
任务六: 综合场景与自由创新	已实现主要演示负载	科研智能体平台同时运行在未修改 uCore 与 AgentOS-uCore 上，增强目标使用可信上下文、文件对象查询、事件通知、权限、审计记录、运行时间线、来源追踪和大模型转发支撑机制。

1.6 项目开发情况

项目在 uCore 基础上完成了内核机制、用户态演示负载、宿主机结果提取和文档材料四部分建设。内核部分集中在智能体进程状态、上下文页、工具运行时、文件对象元数据、事件队列、等待唤醒、调度提示、审计记录、运行时间线和来源追踪。用户态部分包含 AgentOS 专项测试程序、科研智能体平台程序、普通 uCore 对照程序和大模型转发程序。宿主机部分包含镜像状态提取、双目标结果对照、图表生成、状态摘要和一键验证脚本。文档部分围绕系统架构、接口说明、测试说明、演示说明和设计开发报告展开，所有对外材料均以当前仓库中的源码、脚本和可生成结果为依据。

我们已经形成从内核机制到演示材料的完整运行链路。运行双目标脚本后，系统会分别执行普通 uCore 和 AgentOS-uCore，提取状态文件，生成 CSV、SVG 图表、HTML 文件和 Markdown 摘要。报告中的测试数据和图表说明均对应这一产物链路，便于复查输入、运行过程和输出。

1.7 与操作系统内核机制的对应关系

AgentOS-uCore 的核心实现直接嵌入 uCore 的进程、地址空间、系统调用、文件系统、时钟和同步路径。表 4 按内核机制梳理这些实现关系，便于把智能体 workflow 需求和 uCore 原有执行路径联系起来。

表 4: 内核机制与项目实施对应

内核机制	项目中的实现	报告和测试中的证据
启动流程与 QEMU 运行	通过指定初始进程启动专项测试和科研平台程序，脚本统一构建镜像并运行 RISC-V QEMU。	AgentOS 专项测试、双目标运行结果、运行日志和状态摘要。
异常与系统调用处理	新增智能体相关系统调用，入口负责参数读取、地址预检、返回码和错误语义。	工具调用运行时、权限测试、坏参数和非法地址测试。
进程管理	在进程控制块中加入智能体身份、能力位、上下文、心跳、事件和可观测状态。	智能体创建、执行新程序、退出释放、普通进程拒绝测试。
虚拟内存与地址空间	为智能体映射上下文镜像，内核维护权威副本，用户缓存区保留给策略数据。	镜像篡改恢复、用户缓存保留、上下文快照测试。
文件系统与文件描述符	元数据绑定真实文件对象，保护私有后端文件，文件变化时更新对象状态。	文件对象查询、元数据重载、删除清理、私有后端保护测试。
中断与时钟	使用 tick 处理等待超时、心跳事件、调度观察和运行时间线。	事件等待、心跳停止、有限等待睡眠、调度原因测试。
并发同步	事件队列、消息、文件租约和元数据更新使用内核锁保护。	并发写入实验、消息唤醒测试、租约拒绝测试。
内核态与用户态隔离	普通进程不能伪造智能体身份，用户态镜像不能污染内核可信历史。	权限测试、镜像脏写测试、审计记录和来源追踪检查。

这些对应关系也决定了后文的组织方式。第 2 章按内核机制展开，实现细节围绕 uCore 原有路径说明；第 3 章按测试场景展开，每个场景都说明普通路径与 AgentOS 路径在相同输入下的差异。这样的安排使智能体系统需求能够落到具体的进程、内存、文件、同步和调度实现上。

1.8 团队成员分工

团队工作围绕内核机制实现、用户态演示负载、测试脚本、宿主机结果工具和文档材料展开。两名成员的主要分工如下：

表 5: 团队成员主要分工

成员	主要工作
康峻豪	负责 xv6 原型阶段的内核机制开发, 包括智能体进程创建、智能体地址空间和上下文区、结构化工具调用、Context Path、基础演示程序和任务一至任务三相关测试; 负责普通 uCore 下用户态科研 Agent 平台的主要功能开发, 包括 workflow 编排、状态文件组织、角色程序、工件记录、对照运行入口和平台状态输出。
王浩洋	负责将 xv6 原型中的智能体进程、结构化工具调用和 Context Path 等机制迁移并泛化到 uCore/AgentOS-uCore 内核, 继续实现文件对象元数据查询、事件队列、等待唤醒、心跳、权限能力、调度提示、审计记录、运行时间线、来源追踪和文件编辑租约; 负责把科研 Agent 平台接入 AgentOS-uCore 内核能力, 完成双目标测试、文档及说明材料准备和说明视频制作。

项目开发进程如下：

表 6: 项目开发进程

时间	工作内容
5 月 25 日–6 月 12 日	基于 xv6-riscv 内核开展任务一至任务三原型开发, 完成智能体进程创建、地址空间和 Context 映射、用户态 ABI、结构化工具调用、工具结果记录、Context Path、基础演示程序和初步测试。
6 月 12 日–6 月 15 日	继续在 xv6 内核上加固任务一至任务三, 实现批量工具调用、Context snapshot、Context 回滚/清理、压力测试和性能观测; 同时将 xv6 中的智能体进程、结构化工具调用和 Context Path 机制迁移到 uCore 内核, 形成 AgentOS-uCore 的基础版本。

时间	工作内容
6 月 15 日-6 月 23 日	开发用于演示和对照的科研 Agent 平台，并移植到未修改 uCore 内核上运行。该阶段完成用户态工作流、角色程序、状态文件、工件记录、模板大模型转发、宿主机提取工具和普通 uCore 对照路径。
6 月 23 日-6 月 28 日	将用于演示的科研 Agent 平台接入 AgentOS-uCore，围绕内核 Context、文件对象查询、事件通知、失败恢复、权限控制、timeline、audit 和 provenance 完善主流程；同时补充 AgentOS 专项测试、双目标对照实验和图表生成。
6 月 28 日-6 月 30 日	完善设计文档、API 文档、测试说明、报告、场景运行脚本和说明材料，整理图表和测试证据，并完成说明视频制作。

2 模块设计和实现

我们在本章按照内核机制展开，科研平台业务流程只作为使用场景出现。每个模块均围绕三个问题说明：普通 uCore 或普通用户态实现中会遇到什么困难；我们在原有内核结构中增加了哪些状态、接口和检查；测试如何验证这些状态和接口确实参与主流程。这样可以把演示平台的策略和内核机制分开，也使实现说明从进程、内存、文件、同步和调度这些基础路径出发。

从 uCore 原有执行路径看，用户程序通过系统调用进入内核，内核根据进程控制块、页表、文件描述符表和调度状态完成资源访问；程序装载由执行新程序路径完成，父子关系由等待和退出路径维护，定时器中断推动调度和超时处理。该流程适合普通批处理程序和交互式程序，但智能体工作流还需要长期保存多轮行动状态、理解工具请求的结构、按文件对象属性查询候选集、让等待中的智能体按事件醒来，并把工具、文件、事件和大模型请求组织成可复查记录。因此，我们的模块扩展均从 uCore 原有路径进入，保持在教学内核框架内完成。

表 7: 模块设计叙述与验证关系

模块	主要问题	实现与验证重点
智能体进程	普通进程缺少智能体身份、能力和长期运行状态。	在进程控制块中增加身份、能力、上下文、事件和心跳字段，测试创建、执行、退出和异常路径。
结构化工具调用	普通系统调用只表达低层参数，无法描述工具名称、参数键值和结果语义。	建立工具表、参数检查、批量执行和错误码，测试吞吐、坏参数和权限拒绝。

模块	主要问题	实现与验证重点
上下文路径	用户态日志分散，恢复时需要重建多轮执行过程。	使用内核权威历史和用户态镜像记录最近路径，测试快照、淘汰、回滚和篡改恢复。
文件对象查询	路径扫描难以表达对象状态、摘要、依赖和租约。	绑定真实文件对象与元数据，提供候选集和摘要查询，测试扫描数、候选数和冲突拒绝。
事件运行循环	长期智能体轮询状态文件会产生无效读取。	使用事件队列、订阅、睡眠等待、心跳和取消等待，测试超时、唤醒和多智能体消息。
审计与来源追踪	报告产物和行动结果缺少统一证据链。	记录工具、事件、文件对象和大模型请求之间的关系，测试状态摘要和来源查询。

2.1 智能体进程层设计

智能体进程层建立在 uCore 原有进程机制之上。普通进程仍使用原有创建、调度、等待、文件描述符和地址空间管理流程；智能体进程在此基础上增加由内核维护的扩展状态。进程控制块保存智能体编号、角色模板、能力位、上下文区位置、上下文统计、事件队列、订阅条件、心跳信息、调度提示、审计计数和来源关系计数。权限判断直接读取内核中的能力位，授权结果由能力位决定。

创建路径分为默认创建和指定角色创建。默认创建用于兼容普通示例，得到最低权限智能体；指定角色创建会检查调用者是否具有编排能力。角色在实现中只是能力位集合的模板，例如演示中的编排者、调查者、恢复者和哨兵都可以映射为不同的能力集合。内核真正依赖的是能力位，包括上下文读写、文件元数据读写、内容读取、消息发送、事件处理、调度配置、依赖更新、来源读取和大模型转发请求等。

地址空间设计围绕上下文区展开。当前智能体上下文区为 6 页，前部保存头部和最近结果，中部保存最近 128 条上下文记录和细节记录，末尾保留用户自管缓存。内核同时维护权威副本和用户态镜像。工具调用、事件、调度和查询产生的新记录先进入权威副本，再同步到用户态镜像。用户态可以快速读取镜像，也可以在需要可信结果时通过快照接口读取内核整理后的记录。用户自管缓存不会被快照覆盖，适合保存智能体的短期策略状态。

执行新程序和退出时，内核会重新建立或释放上下文页，清理事件队列和可观测状态，防止旧进程残留影响新程序。普通进程没有智能体上下文映射，也没有智能体元数据，因此直接写用户地址空间不会获得智能体权限。

2.1.1 进程状态和生命周期

智能体进程状态直接附加在 uCore 的进程控制块中。实现时沿用原有进程生命周期，并增加一组只由内核修改的字段：智能体标志、能力位、角色模板、上下文页内核地址、事件队列、订阅列表、心跳参数、运行统计、审计游标和来源追踪计数。这样设计的好处是保留 uCore 原有的 `fork`、`exec`、`wait`、`exit` 和调度流程，同时让智能体扩展状态随进程一起创建、切换和销毁。

创建智能体时，内核先检查调用者权限，再分配进程和上下文页。若调用者是普通进程，只能创建最低权限智能体；若调用者具有编排能力，可以指定角色模板。创建失败时，内核不会留下半初始化的上下文页或事件队列。执行新程序时，系统重新安装上下文映射并重建头部元信息；若执行失败，旧地址空间和旧上下文仍保持可用。退出时，系统释放上下文页、清空事件队列、取消心跳和等待状态，并把必要统计写入审计记录。

表 8: 智能体进程状态要点

状态类别	主要内容
身份状态	是否为智能体、智能体编号、角色模板、能力位和资源配额。
上下文状态	上下文版本、页数、用户态镜像地址、内核权威副本、用户自管缓存位置。
运行状态	最近工具调用、调用次数、错误次数、事件等待状态、心跳间隔和最近心跳 tick。
协作状态	消息队列、事件队列、订阅条件、调度提示、租约和被拒绝操作统计。
证据状态	上下文顺序号、审计序列、时间线游标、来源关系数量和最近完整性摘要。

2.1.2 地址空间处理

智能体上下文区固定映射到用户地址空间高位区域，普通进程不会自动拥有该映射。上下文区使用 6 页：头部和最近结果位于前部，历史记录和细节记录位于中部，末尾保留用户自管缓存。内核权威副本不依赖用户态镜像的内容，所有可信写入先进入内核副本，再同步到镜像页。用户态可以直接读取镜像提升观察效率，但需要审计或跨进程复核时应使用快照接口。

这种布局处理了两个常见问题。第一，普通进程不能通过伪造地址写入获得智能体身份，因为身份和能力位只在进程控制块中保存。第二，用户态可以污染自己

的镜像页，但无法改变内核权威历史；下一次快照会把镜像恢复为内核副本的内容，用户自管缓存区则不会被覆盖。专项测试会主动写脏镜像区和用户缓存区，分别验证可信历史可恢复、用户缓存可保留。

2.1.3 异常处理和测试证据

智能体进程层的错误处理遵循“失败不产生业务副作用”的原则。创建失败不会分配可见子进程；执行新程序失败不会丢失旧上下文；非法输出地址不会先执行工具；普通进程调用智能体专用接口返回失败；低权限智能体创建高权限角色会被拒绝。对应证据主要来自 `agentfinal_ucore`、`agentsecurity_ucore` 和 `agentsched_ucore`。其中 `agentfinal_ucore` 验证上下文映射和镜像恢复，`agentsecurity_ucore` 验证身份伪造无效和能力授权，`agentsched_ucore` 验证调度原因和预算状态可以从内核读取。

2.1.4 与 uCore 原有机制的关系

AgentOS-uCore 在 uCore 原有进程机制上增加智能体语义。创建智能体仍然需要经过进程分配和地址空间建立；执行新程序仍然使用 uCore 的程序装载路径；等待和退出仍然遵守父子进程关系；调度仍然由内核调度器选择可运行进程。智能体进程额外拥有一组由内核维护的语义状态，调度器和系统调用入口能够读取这些状态，进而完成权限检查、上下文记录、事件等待和可观测信息更新。

这种设计更适合教学操作系统。它保留了 uCore 原有实验结构，便于说明 `fork`、`exec`、`wait`、地址空间和调度过程；同时又能体现智能体工作流对操作系统的新需求。表 9 给出主要接入点。每一项接入都对应明确的测试：进程创建和执行失败由专项测试覆盖，地址空间由上下文镜像和缓存测试覆盖，文件系统由文件对象查询测试覆盖，时钟中断由等待和心跳测试覆盖。

表 9: AgentOS-uCore 对 uCore 机制的接入方式

uCore 机制	AgentOS-uCore 扩展	验证重点
进程控制块	增加智能体身份、能力位、事件、心跳、时间线和来源计数。	普通进程不能伪造身份，低权限智能体不能越权。
程序装载	智能体执行新程序时重建上下文映射，失败时保留旧状态。	执行失败后仍能读取信息、调用工具和查询上下文。

uCore 机制	AgentOS-uCore 扩展	验证重点
虚拟内存	在高地址映射上下文镜像，内核保留权威副本和用户缓存位置。	镜像篡改不会污染可信历史，用户缓存不会被快照覆盖。
系统调用	增加智能体创建、工具调用、上下文、文件查询、事件等待等接口。	参数错误、非法地址、权限不足均返回明确状态。
文件系统	文件对象元数据绑定真实文件对象，支持摘要、索引和租约。	文件创建、删除、重载和冲突写入都有可复查结果。
时钟与调度	利用 tick 处理超时、心跳、等待唤醒和调度原因记录。	有限等待进入睡眠，心跳停止后不再生成可消费事件。

2.2 结构化工具调用运行时

工具调用运行时把智能体的系统交互从“字符串命令”提升为“结构化请求”。请求包含工具名称或编号、请求编号、数值参数、文本参数和参数键名；结果包含状态码、返回值、结果摘要和错误说明。内核工具表记录每个工具的参数类型、是否允许批量调用、是否只能通过专门系统调用执行。这样做可以在执行前发现错误参数、错误类型和越权请求，避免失败请求写入上下文、发送消息或修改文件元数据。

该模块解决的是“行动语义”表达问题。普通系统调用只告诉内核读写哪个地址、打开哪个文件或等待哪个对象，无法说明这次行动在智能体流程中代表查询、恢复、通知、产物更新还是大模型请求。AgentOS-uCore 的工具运行时把行动拆成工具编号、工具名称、参数槽、文本摘要、返回状态和结果摘要。内核并不理解科研平台中某个阶段的业务含义，但能够理解该行动属于文件查询、事件投递、上下文读取、对象更新或大模型请求，从而进行统一的权限检查和记录。

接口分为两类。按名称调用保留“工具名称 + 参数键值列表”的形式，适合可读性较高的调用位置。按编号批量执行服务高频路径，一次系统调用最多执行 64 个工具操作，减少系统调用次数和重复校验开销。两个接口最终进入同一套执行逻辑，因此返回码、权限检查和上下文记录保持一致。

表 10: 代表性接口与用途

接口	用途
agent_create_role	创建指定能力模板的智能体进程，内核按调用者能力决定是否允许。
agent_call	按工具名称和参数键值列表执行一次结构化工具调用。
agent_run	按工具编号批量执行工具，面向高吞吐路径。
context_snapshot	读取内核整理后的可信上下文快照。
agent_file_query	按文件对象元数据或路径条件查询候选文件。
agent_wait	智能体进入事件等待，直到事件到达、超时或被取消。
agent_llm_request	记录大模型请求并把请求投递给具备转发能力的进程。

内置工具保持通用语义，例如回显、进程信息、系统状态、文件查询、内容摘要、消息发送、上下文统计、事件投递、通用动作提交、产物更新、依赖更新、大模型请求和大模型响应。科研平台中的“重新执行某个阶段”“写入报告状态”等业务动作由用户态组合通用动作、产物更新、文件元数据和事件通知完成。

2.2.1 调用路径

工具调用分为用户态封装、系统调用入口、工具表解析、参数校验、权限检查、工具执行、结果写回和证据记录八个步骤。按名称调用先根据工具名查表，并检查参数键名和类型；按编号批量调用直接使用工具编号定位工具，再校验编号、名称和参数槽。批量接口会先检查整批输入和输出地址是否可访问，确保地址错误时不会执行前面的工具，也不会向上下文或消息队列写入半截结果。

一次工具执行完成后，内核会把结果写入用户提供的结果数组，同时写入最近结果区、上下文记录、时间线和审计记录。若工具产生文件对象变化、事件或来源关系，还会调用对应子系统追加记录。这样的执行路径保证了同一工具在单次调用和批量调用下具有一致语义，也保证用户态看到的结果、上下文和审计记录来自同一内核执行点。

表 11 展示一次工具调用在内核中的主要阶段。它对应报告第 3 章中的吞吐实验：单次调用每次都要走完入口，批量调用把多个工具放进同一次系统调用，减少重复进入内核、重复查表和重复同步上下文的成本。在工具数量较少时，二者差距不大；当智能体连续进行文件查询、上下文读取和事件投递时，批量路径的优势会更明显。

表 11: 结构化工具调用执行阶段

阶段	内核处理	产生的可观测结果
入口预检	检查调用者是否为智能体，检查输入和输出用户地址。	地址错误时不执行工具，不增加调用次数。
工具解析	按名称或编号定位工具，读取工具属性和参数规则。	未知工具返回参数错误，不写业务状态。
参数校验	检查键名、类型、长度、枚举值和 selector 格式。	错误摘要写入返回结构，便于用户态显示原因。
能力检查	根据进程控制块中的能力位判断是否允许执行。	拒绝记录进入审计，用户态自报角色无效。
工具执行	查询进程、文件对象、上下文、事件、消息或大模型请求。	结果写入返回数组和最近结果区。
证据追加	写入上下文、时间线、审计记录 and 来源关系。	状态摘要能从内核证据中复原执行过程。

2.2.2 参数和返回码

工具运行时使用统一返回码表达结果。普通进程调用智能体专用接口返回失败；参数键名、类型、长度或枚举值错误返回参数错误；能力不足返回拒绝；目标对象不存在返回未找到；超时、重复请求、空间不足和队列满分别使用专门状态。每个错误状态都会尽量写入可读摘要，便于用户态显示原因。

表 12: 工具调用返回语义

状态	含义
成功	工具完成，结果值和摘要有效，相关上下文和审计记录已追加。
参数错误	工具编号、名称、参数键名、参数类型、字符串长度或 selector 格式不符合工具描述。
权限拒绝	当前进程的能力位不足，用户态传入角色值不会改变授权结果。
未找到	目标进程、文件对象、依赖记录、事件、来源记录或可见历史节点不存在。
重复请求	带有相同幂等键的动作已经成功提交，内核拒绝重复修改。

状态	含义
超时	等待事件或大模型响应超过指定 tick，状态进入可查询记录。
空间不足	上下文布局、事件队列或记录区无法容纳本次写入。

2.2.3 权限语义

工具表只描述“这个工具需要什么能力”，不描述某个科研角色的业务含义。调用时内核读取当前进程控制块中的能力位，决定是否允许文件元数据写入、内容读取、事件投递、依赖更新、产物写入、大模型转发或审计读取。用户态可以把角色名称作为展示字段，也可以在演示中称某个智能体为恢复者或审计者，但授权只看能力位。测试程序会让低权限智能体伪造更高角色，再调用受保护工具，期望结果仍为权限拒绝。

权限语义还服务任务六的综合演示。科研平台中的调查者可以读取文件摘要和上下文，恢复者可以提交通用动作和更新产物，写作者可以写入报告状态，审计者可以读取审计记录。内核并不关心这些角色名称，只检查对应能力位。这样处理后，演示中的角色分工仍然清晰，内核机制也不会被某一个科研流程固定住。

2.3 上下文路径与可信历史

上下文路径记录智能体多轮工具调用和事件处理过程。每条记录包含顺序号、请求编号、原因记录、跨度编号、时间、工具、状态、参数摘要、结果摘要和完整性摘要。记录容量固定为 128 条，写满后按先进先出方式淘汰，头部统计最早顺序号、最新顺序号、已淘汰数量和回滚次数。细节记录保存更完整的摘要信息，供可视化材料和来源追踪使用。

Listing 1: 上下文记录实现摘录

```
1 struct agent_context_record {
2     uint64 sequence;
3     uint64 request_id;
4     uint64 cause_sequence;
5     uint64 span_id;
6     uint64 tick;
7     uint64 prev_hash;
8     uint64 record_hash;
9     int tool_id;
10    int status;
11    char payload[AGENT_CONTEXT_TEXT_SIZE];
```

```
12     char result[AGENT_CONTEXT_TEXT_SIZE];
13 };
```

上下文路径有两条读取方式。快速方式是用户态直接读取镜像区，适合获取最近状态和策略缓存。可信方式是通过快照接口读取内核权威副本，适合生成审计材料、演示证据和跨进程复核。测试中会故意篡改用户态镜像，再通过快照接口刷新并确认可信结果未受影响。

上下文路径还与运行时间线和来源追踪共用原因记录和跨度编号。一次工具调用可以派生时间线记录，也可以生成来源关系边；一次事件唤醒可以关联到触发它的消息、文件对象或大模型响应。这样，用户态平台读取内核证据时能够回答“这个结果从哪里来、经过了哪些工具、由哪个事件触发、是否具有对应权限”。

上下文区采用“内核权威副本 + 用户态镜像 + 用户缓存”的三段思路。内核权威副本保证可信历史，用户态镜像减少高频读取的系统调用，用户缓存允许智能体保存短期策略状态。三者的用途不同，避免把所有状态都塞进可信历史，也避免让可信历史依赖用户可写内存。表 13 给出各区域的作用。

表 13: 智能体上下文区组成

区域	主要内容	设计目的
头部和最近结果	版本、记录数量、顺序号、最近工具结果、缓存位置。	让用户态快速判断上下文状态。
可信历史	最近 128 条上下文记录和细节记录。	支持审计、恢复、来源查询和跨进程复核。
用户态镜像	从内核权威副本同步出的可读视图。	减少频繁读取上下文时的系统调用成本。
用户自管缓存	智能体策略缓存、短期分 值、可视化辅助数据。	给用户态保留可写空间，不进入内核可信历史。

2.3.1 写入规则

上下文记录由两类来源产生。第一类是自动记录，工具调用、事件消费、调度提示、文件对象查询、大模型请求和大模型响应在执行完成后自动追加。第二类是手动追加，用户态可以通过上下文接口写入摘要节点，用于标记计划、阶段说明或中间判断。两类记录共用同一顺序号，因而能够按时间顺序还原同一个智能体的执行路径。

写入记录时，内核先生成新的顺序号和跨度编号，再计算前一条记录的摘要，最后把参数摘要、结果摘要、状态和 tick 写入权威副本。若记录来自工具调用，工具

编号和请求编号会一并保存；若记录来自事件，事件类型和事件来源会进入细节记录；若记录来自大模型转发，请求编号和响应摘要会和来源关系一起保存。用户态镜像只在内核副本写入完成后同步。

2.3.2 查询和淘汰

上下文容量固定为 128 条最近记录，超出后按先进先出方式淘汰。头部记录最早顺序号、最新顺序号、当前记录数和淘汰数量，用户态可以判断某个旧记录是否已经不可见。查询接口支持从指定顺序号开始读取可见记录；快照接口一次返回头部和按顺序排列的记录，适合生成结果材料和审计材料；回滚接口只允许回滚到当前可见的历史节点。

这种设计把内核保存范围限定为最近路径、短摘要和必要的因果信息。完整报告、长文本和可视化内容仍由用户态文件系统保存；内核保存足以支持决策、追踪和验证的结构化摘要。测试中通过 128 条以上写入验证淘汰计数，通过回滚不存在节点验证未找到返回码，通过直接篡改镜像验证快照可恢复可信记录。

表 14: 上下文路径测试证据

测试点	验证内容
镜像读取	用户态可直接读取最近头部、结果和记录，用于高频读取。
可信快照	快照接口返回内核权威副本，用户态改写镜像后仍可恢复。
容量淘汰	写入超过 128 条后，最早顺序号和淘汰数量按预期变化。
短文本摘要	不同工具 payload 和 result 的短摘要可在快照中恢复。
用户缓存	用户自管缓存不进入可信历史，快照不会覆盖该区域。

2.4 文件对象元数据与内容摘要查询

任务四要求系统支持面向智能体的文件查询。AgentOS-uCore 采用文件对象元数据服务实现该能力。每条记录绑定真实文件对象，保存物理文件名、逻辑路径、项目名、工作流名、运行编号、阶段、类型、状态、摘要、依赖标记、设备号、节点号、大小和版本信息。字段名称兼容科研平台，但内核只把它们当作通用标签、状态和摘要，用户态可以把同一机制用于代码文件、日志文件、记忆文件、报告文件、配置文件或数据文件。

元数据后端是内核私有文件，普通文件接口会拒绝打开、创建、截断或删除该后端。智能体子系统通过内核辅助函数加载和写回记录，文件创建、写入、截断和删除时同步更新相关状态。查询时，系统根据条件选择扫描路径或索引路径，并返回

查询计划、候选记录数、实际触达记录数和命中结果。用户可以从结果中看到本次查询是完整扫描，还是先由状态、阶段、类型等标签缩小候选集。

内容摘要查询补足了仅靠属性查询的不足。智能体可以读取文件预览、内容摘要和版本信息，用于判断缓存是否有效，或把摘要写入上下文路径。预取提示利用依赖记录推断下一步可能访问的对象，提前把候选对象写入运行证据。文件编辑租约用于多智能体协作：写入者需要先获得租约，提交时检查持有者、版本和权限，过期或越权写入会被拒绝并写入审计记录。

2.4.1 元数据模型

文件对象元数据记录被设计为通用标签集合。科研平台使用其中的项目、 workflow、运行编号、阶段和报告状态字段；其他应用可以把同一组字段解释为命名空间、对象编号、所有者、对象类型、状态、优先级、版本、标签、策略标记和内容摘要。内核只负责保存、查询和更新这些通用字段，不把某个实验流程写成固定逻辑。

表 15: 文件对象元数据字段分组

字段分组	用途
身份字段	物理文件名、逻辑路径、设备号、节点号、对象编号和命名空间。
分类字段	类型、阶段、状态、标签、策略标记、优先级和所属 workflow。
内容字段	文件大小、版本、内容摘要、预览摘要和更新时间。
关系字段	依赖键、来源对象、关联请求、最近产物和来源追踪编号。
控制字段	租约持有者、租约到期 tick、更新掩码和查询缓存状态。

2.4.2 查询流程

查询从 selector 解析开始。若请求只包含路径，系统直接定位文件对象并返回摘要；若请求包含标签、状态、类型或命名空间，系统先使用内存索引选出候选，再按真实文件状态复核；若条件过宽或索引不可用，系统回到扫描路径，并在结果中记录触达记录数。无论采用哪条路径，返回结果都会包含查询计划、候选数量、扫描数量和命中数量，用户态工具可以据此解释本次查询成本。

元数据后端文件只由 Agent 子系统内部读写。普通进程对后端文件执行打开、创建、截断或删除会失败，避免用户态绕过内核工具直接篡改 metadata。重载测试会先写入自定义记录，再重新初始化元数据服务，确认系统从后端文件加载已有记

录，演示默认数据不会覆盖已经存在的记录。

文件对象查询的核心价值在于让智能体先使用语义条件缩小候选集，再读取必要内容。普通用户态平台如果要寻找“本次运行中失败且需要恢复的报告对象”，通常需要遍历状态目录，解析文件名和内容字段。AgentOS-uCore 可以把状态、类型、运行编号、依赖键和版本作为对象属性，由内核返回候选数量、扫描数量和命中数量。第 3 章的文件对象实验使用 32 到 1024 个文件对象，呈现索引候选集随规模增长保持较小，而普通扫描数随文件数线性增长。

表 16: 文件对象查询处理步骤

步骤	处理内容	输出信息
条件解析	读取路径、状态、类型、命名空间、标签和依赖键。	判断能否使用索引路径。
候选选择	使用内存索引选出可能命中的对象。	返回候选记录数,说明缩小范围的效果。
真实复核	检查文件对象仍存在、版本有效、摘要可用。	返回扫描数量和命中数量。
结果记录	把查询计划、候选数量和摘要写入上下文。	后续智能体可以复用本次查询证据。
预取提示	根据依赖记录推断下一步可能访问的对象。	进入运行时间线和来源追踪。

2.4.3 租约和冲突处理

多智能体写同一对象时，普通用户态常用锁文件和约定字段协调。AgentOS-uCore 把租约检查放到内核：申请租约时记录持有者、对象版本和过期 tick；提交写入时检查调用者是否为持有者、版本是否匹配、租约是否过期、能力位是否允许写入。失败写入不会修改对象状态，只增加拒绝计数并写入审计记录。这样，演示平台中的报告写入、恢复状态更新和 artifact 更新可以得到统一保护。

2.5 事件驱动的智能体运行循环

长期运行智能体需要避免不停读取状态文件来等待工作。AgentOS-uCore 为每个智能体维护事件队列和订阅列表。智能体先注册感兴趣的事件类型和过滤条件，再进入等待；当消息到达、文件对象变化、心跳触发、大模型响应返回或用户态显式投递事件时，内核把事件放入队列并唤醒目标智能体。事件队列有容量限制，队列满会增加丢弃计数，便于测试观察压力情况。

等待逻辑采用真实睡眠。有限超时会设置截止时间，然后让出 CPU；时钟到达后由内核唤醒并返回超时事件。心跳通过同一事件系统投递定时事件，停止心跳后停止产生可消费事件。取消等待会生成取消事件并唤醒目标智能体。多智能体消息使用目标进程锁保护读写，避免发送者和接收者在多核 QEMU 下出现竞态。

Listing 2: 事件等待实现摘录

```
1 if (agent_event_dequeue(p, &event)) {
2     return event.status;
3 }
4 if (deadline_reached(p)) {
5     fill_timeout_event(&event);
6     return event.status;
7 }
8 p->agent_wait_sleep_count++;
9 sched();
```

调度提示记录智能体被唤醒和被调度的原因，包括事件队列长度、角色权重、截止时间、预算使用、心跳状态和最近事件。专项测试会读取这些状态，用数据确认智能体运行循环已经进入内核管理。

2.5.1 事件类型和队列行为

事件队列保存固定数量的最近事件，事件来源包括消息、文件对象变化、心跳、超时、取消等待、大模型响应、通用动作提交和用户态显式投递。订阅列表保存智能体愿意接收的事件类型和过滤条件，未订阅事件不会被作为可消费事件投递。队列满时系统丢弃新事件或记录丢弃次数，具体行为由事件类型和调用者能力决定；无论采用哪种处理，计数都会进入可观测状态。

表 17: 智能体运行循环中的关键状态

状态	说明
订阅列表	最多保存若干 watch 条件，决定哪些事件会进入可消费队列。
事件队列	保存待消费事件，记录入队、出队、丢弃和最近事件类型。
等待状态	保存是否正在等待、截止 tick、等待原因和睡眠次数。
心跳状态	保存心跳间隔、最近心跳 tick、停止状态和定时事件数量。
唤醒统计	保存消息唤醒、定时唤醒、取消唤醒和超时唤醒次数。

2.5.2 与普通轮询的差异

普通用户态平台为了等待结果，通常需要反复读取状态文件或日志。事件数量增加时，轮询次数会按检查频率增长，与实际事件数并不一致。AgentOS-uCore 的等待路径让智能体进入睡眠，只有事件入队、心跳到期、取消等待或超时到达时才重新运行。测试通过读取等待循环计数、睡眠计数、唤醒计数和超时计数，确认有限超时不会反复占用 CPU。

事件运行层与 uCore 的时钟中断和调度状态紧密相关。有限等待会记录截止 tick，随后进入睡眠；时钟处理逻辑发现截止 tick 到达时唤醒目标进程并写入超时状态。心跳也是通过 tick 推进，只有订阅了对应事件的智能体才会收到可消费事件。消息投递和事件入队使用目标进程锁保护队列状态，保证发送者和接收者不会同时改写同一队列。由此，AgentOS-uCore 可以把“长期等待任务”转化为普通内核睡眠和唤醒问题，减少用户态反复读取文件的需要。

表 18: 事件等待与调度相关状态

状态	更新时机	用途
等待原因	调用等待接口时写入。	说明智能体等待的是消息、文件变化、定时器还是大模型响应。
截止 tick	有限等待或大模型请求设置。	时钟处理逻辑据此判断是否返回超时。
睡眠计数	进程真正进入等待时增加。	确认系统没有用用户态循环替代等待。
唤醒原因	事件入队、心跳、取消等待或超时写入。	调度测试和可视化材料可说明为什么恢复运行。
丢弃计数	队列满或未订阅事件被过滤时更新。	在压力测试中观察事件容量是否足够。

2.6 审计记录、运行时间线、来源追踪与大模型友好机制

AgentOS-uCore 的可观测层由三类记录组成。审计记录用于保存关键行为，例如上下文追加、事件入队、事件消费、调度、预取提示、权限拒绝和文件编辑租约结果。运行时间线把不同来源的记录整理成统一时间序列，支持按来源、时间、智能体、工具、事件和状态查询。来源追踪把工具调用、文件对象、大模型请求、事件和产物之间的关系表达为可导出的边，用户态平台可以把它们导出为可视化证据。

大模型友好机制遵循明确分工。内核记录大模型请求和响应的结构化信息，包括请求编号、目标转发进程、提示摘要、响应摘要、超时、状态、触发事件和来源关

系；云端 API、密钥、网络协议和模型策略由用户态或宿主机转发层实现。默认测试使用模板转发，不需要密钥；宿主机具备密钥时可以切换到真实云端调用。无论采用哪种转发方式，内核看到的都是结构化请求、事件和审计记录。

2.6.1 记录组织方式

审计记录强调“谁做了什么、是否成功、为什么被允许或拒绝”。运行时间线强调“什么时候发生、由什么触发、影响了哪个对象”。来源追踪强调“一个结果由哪些输入、工具、事件和大模型响应产生”。三类记录使用共同的顺序号、tick、智能体编号、工具编号和跨度编号，用户态可以按不同视角读取同一批运行事实。

表 19: 可观测记录和使用场景

记录类别	使用场景
审计记录	查看权限拒绝、租约结果、上下文写入、事件投递和关键工具执行结果。
运行时间线	按时间记录智能体创建、工具调用、文件查询、等待唤醒、调度和大模型响应。
来源追踪	把文件对象、工具调用、事件、产物和大模型请求组织成有向关系，服务报告解释。
摘要统计	给出记录数量、最近 tick、失败次数、拒绝次数、游标位置和完整性摘要。

2.6.2 大模型请求处理

大模型请求并不意味着内核直接访问云端。AgentOS-uCore 的处理流程是：请求智能体向内核提交结构化请求，内核检查调用者能力和预算，记录请求摘要并向转发进程投递事件；用户态转发进程读取请求，使用模板模式或宿主机云端模式生成响应；响应回到内核后，系统写入上下文、审计记录、运行时间线和来源关系，并唤醒请求智能体。这个流程让大模型调用进入操作系统可观察范围，同时不把密钥、HTTP、TLS 和具体模型端点放进教学内核。

2.6.3 测试证据

`agentllm_ucore` 覆盖模板转发路径，检查请求智能体、转发智能体和响应事件之间的关系；宿主机 LLM Relay 测试覆盖默认模板模式和可选云端模式，默认情况下无需密钥即可复现；双目标实验中的 LLM Relay 组比较请求数增长时普通路径重

建日志的步骤与 AgentOS 结构化记录数量，说明大模型相关行为可以纳入统一证据链。

可观测层的测试不只判断接口是否返回成功，还要检查记录之间能否互相印证。例如，一次大模型请求应当在请求方上下文中出现，在转发进程事件中出现，在审计记录中出现，并在来源追踪中连接输入文件、提示摘要、响应摘要和最终产物。若其中某一类记录缺失，报告和状态摘要就无法解释结论来源。第 3 章的双目标对照和 LLM Relay 实验正是围绕这些证据关系设计。

2.7 关键设计取舍

在 AgentOS-uCore 的实现过程中，我们有几项需要明确说明的设计取舍。这些取舍直接影响系统的通用性、性能和可复查性，也说明科研平台在我们的工作中承担演示负载的角色，内核机制保持通用。

第一，内核保存通用机制，不保存具体科研策略。内核认识智能体身份、能力位、上下文、文件对象、事件、动作、产物、审计和来源关系；内核不认识某个具体项目名、固定运行编号或固定阶段顺序。科研平台需要的阶段依赖、报告状态和恢复动作由用户态通过通用接口写入。这样，项目可以用同一套机制支撑科研流程，也可以支撑代码修复、运维巡检或数据处理流程。

第二，大模型请求进入系统管理视野，但云端服务不进入内核。内核保存请求编号、预算、提示摘要、响应摘要、超时、事件和来源关系；网络连接、密钥、模型名称和提示词策略由用户态或宿主机转发层负责。这样既能体现 LLM 工作流对操作系统的新需求，又不会把教学内核变成 HTTP/TLS 客户端。

第三，上下文区采用内核权威副本和用户态镜像并存的方式。完全依赖用户态日志，可信性不足；所有查询都走系统调用，频繁读取成本较高。当前设计让可信历史保存在内核，镜像页服务高频读取，用户缓存服务策略状态。三者各司其职，测试也分别验证镜像篡改、快照恢复和用户缓存保留。

第四，文件查询选择对象元数据和内容摘要。uCore 教学文件系统容量有限，项目目标是验证 Agent 友好的文件对象查询机制，因此系统重点支持状态、类型、命名空间、版本、摘要、依赖和租约。完整全文索引可以放在用户态平台或宿主机工具中；内核提供可解释、低成本、可审计的候选集和摘要路径。

第五，事件等待使用内核睡眠和唤醒。智能体长期运行时，普通轮询会不断读文件或查状态；事件队列和订阅列表让智能体在无事件时让出 CPU，事件到达、心跳到期、取消等待或超时时再恢复运行。该设计直接对应任务五，也能服务 LLM Relay、文件变化通知和多智能体消息。

表 20: 主要设计取舍与理由

设计点	采用方案	理由
业务语义位置	科研策略放在用户态，内核只保留通用对象和动作。	保证内核可支持多类智能体应用。
大模型调用	用户态或宿主机执行云端调用，内核记录结构化请求。	避免在教学内核中保存密钥和网络协议。
上下文保存	内核权威副本、用户态镜像和用户缓存组合。	同时满足可信记录、高频读取和策略状态需求。
文件查询	元数据、摘要、索引候选和真实文件复核。	在 uCore 条件下提供可解释查询路径。
事件运行	订阅、睡眠等待、唤醒、心跳和取消等待。	减少用户态轮询，保留可观测等待状态。
权限控制	以能力位作为授权依据，角色只是模板。	防止用户态伪造角色影响内核判断。
测试结果	双目标运行、CSV、SVG 图表和状态摘要。	让功能、性能观察和结果材料来自同一产物。

2.8 科研智能体平台演示负载

科研智能体平台作为任务六和双目标对照的综合负载。未修改 uCore 目标运行普通用户态平台，通过状态文件、日志和普通进程完成同一批科研请求。增强目标运行接入 AgentOS 的平台，把关键阶段绑定到智能体创建、上下文读取、文件对象查询、事件通知、权限检查、来源追踪和审计记录。两条路径共享输入数据和演示请求，因此可以比较同一工作负载在普通机制和 AgentOS 机制下的差异。

科研平台在增强目标上的关键流程可以概括为：编排智能体接收宿主机预置请求，写入演示对象元数据和依赖记录；检索智能体按对象属性读取输入摘要；分析智能体提交结构化工具调用和大模型请求；复核智能体读取上下文与来源关系；恢复智能体在失败阶段提交通用恢复动作；写作智能体更新报告产物状态；审计智能体读取时间线和审计记录。每一步业务策略都在用户态决定，内核负责身份、权限、上下文、事件和证据。

这种拆分使科研平台既能承担赛题任务六的综合演示，又不会把内核设计限制在某个科研项目中。换成代码仓库修复、系统运维巡检或数据流水线检查时，用户态只需替换对象元数据、工具组合和结果呈现；内核的上下文、文件查询、事件等

待、权限和来源追踪机制仍然可用。

3 项目测试结果

3.1 测试体系设计

我们把测试分为内核专项测试、双目标科研平台测试和宿主机结果检查。专项测试由脚本串起，每个测试以一个 uCore 初始进程形式在 QEMU 中运行，输出通过标记后脚本关闭 QEMU。双目标测试分别启动未修改 uCore 和 AgentOS-uCore，运行同一批科研平台请求，再提取文件系统镜像中的状态文件。宿主机结果检查会核对 CSV、图表、JSON、状态摘要和大模型转发结果，保证结果材料来自实际运行数据。

为贴近智能体 workflows 中的真实成本，我们按四类高频场景设置实验负载：文件对象数量从 32 增加到 1024，用来观察普通路径扫描量和内核元数据候选集的差异；上下文与运行记录数量从 128 增加到 8192，用来观察用户态重建步骤和内核快照查询成本；事件数量从 8 增加到 512，用来观察轮询等待和睡眠唤醒的差异；并发智能体数量从 2 增加到 16，用来观察锁文件方案与内核租约、能力检查对覆盖风险的处理效果。大模型转发和失败恢复实验在此基础上补充端到端场景，验证结构化记录和来源追踪能否进入科研平台主流程。

我们在本章按“场景、对照、参数、产物、现象解释”的顺序展开测试说明。场景说明该实验模拟智能体 workflows 中的哪一类真实成本；对照说明普通用户态路径与 AgentOS 路径分别如何完成同一任务；参数说明文件数、记录数、事件数、请求数或智能体数量如何变化；产物说明原始 CSV、统计 CSV、QEMU 输出和状态摘要从哪里来；现象解释说明曲线变化与内核机制之间的关系。这样的写法比单纯列出通过标记更适合评估系统设计，因为它能够呈现同一负载下系统行为是否真的发生变化。

我们在测试设计中强调同一负载下的差异。文件对象查询实验比较普通扫描量与内核元数据候选集；上下文实验比较用户态重建步骤与内核快照查询；事件实验比较用户态轮询次数与内核等待唤醒次数；并发写入实验比较锁文件方案与内核租约和能力检查。图表只围绕这些核心对照展开，避免用大量无关数量堆叠测试材料。

测试环境以 uCore/QEMU 为基础，因此我们把实验重点放在结构化指标和同环境相对差异上。QEMU 绝对运行秒数会受到宿主机调度、文件系统缓存和终端输出影响，不适合单独作为性能承诺；扫描记录数、候选记录数、上下文重建步骤、轮询次数、拒绝次数和工具调用 ticks 则能够稳定反映系统行为。报告中的图表均由同一次结果目录生成，可从统计表回到原始 CSV，再回到 QEMU 输出中的通过标记。

表 21: 六组成组对照实验设计

实验	负载变化	主要指标	说明
文件对象查询	32、128、512、1024 个文件对象	扫描记录数、候选记录数、tick	验证元数据索引是否减少文件扫描。
上下文与时间线	128、512、2048、8192 条记录	重建步骤、快照查询成本、P95	验证内核快照和游标查询是否减少重建工作。
事件等待	8、32、128、512 个事件	轮询次数、wait-/wake 次数、P50/P95	验证睡眠等待是否替代重复轮询。
并发写入	2、4、8、16 个智能体	残余风险、拒绝效果、tick	验证租约和能力检查是否降低覆盖风险。
LLM Relay	4、16、64、256 个请求	重建步骤、结构化记录数、tick	验证大模型请求是否进入统一证据链。
恢复流程	1、3、6、12 个失败阶段	用户态恢复步骤、结构化动作成本、P95	验证恢复动作、幂等和来源追踪是否减少重复拼接。

每组实验都输出 raw CSV、统计 CSV 和机制说明 CSV。raw CSV 保存每次运行的负载、路径、主要指标和 tick；统计 CSV 给出 min、avg、max、P50 和 P95；机制说明 CSV 写明普通路径、AgentOS 路径和设计解释。宿主机测试会检查六个 raw CSV、十一张核心 SVG、HTML 链接和图中文字位置，确保图表既能进入报告，也能直接用于结果材料。

完整验证产物位于运行结果目录。该目录包含普通目标和增强目标的状态提取结果、汇总 CSV、实验设计、证据清单、演示检查表、图表和 Markdown 摘要。报告中的数值来自这些产物，不单独手写无法追溯的结果。表 22 给出主要文件及用途。

表 22: 测试结果产物与报告引用关系

产物	用途
摘要结果	保存双目标状态摘要、API 数量、QEMU 运行诊断和 AgentOS 证据数量。
实验设计	保存六组实验的负载、对照路径、指标定义和解释文字。
原始 CSV	保存每组实验每次运行的原始记录，用于复查具体负载下的数值。

产物	用途
统计 CSV	保存 min、avg、max、P50 和 P95，是本章统计表的直接来源。
证据清单	保存 Context、文件对象查询、事件、审计、来源追踪、权限和时间线等证据命中情况。
图表文件	保存十一张实验图表，供报告和结果材料共用。

演示时核心命令完成构建、双目标运行、状态提取和图表生成。报告使用同一环境、同一输入下的相对差异以及扫描数、记录数、等待次数、拒绝次数等结构化指标，具体机器上的绝对耗时只作为运行观察。

3.2 AgentOS 专项测试

表 23: AgentOS-uCore 专项测试入口

测试程序	覆盖内容	通过标记
agentfinal_ucore	智能体创建、上下文映射、批量工具调用、用户自管缓存、按名称工具协议、通用动作、产物更新、大模型模板转发、运行时间线和来源追踪。	parent passed
agentfs_ucore	真实文件绑定、元数据重载、扫描和索引一致性、查询计划、内容摘要、缓存、字段清空、删除清理和依赖更新。	parent passed
agentscan_ucore	根目录文件扫描、文件创建自动元数据、文件删除自动清理。	parent passed
agentloop_ucore	事件队列、多项订阅、取消订阅、睡眠等待、定时事件、停止心跳和取消等待。	parent passed
agentsched_ucore	调度配置、角色权重、事件优先、调度原因、预算和运行时间统计。	parent passed
agentconflict_ucore	文件编辑租约、持有者提交、非法写入拒绝、过期和版本检查。	parent passed
agentllm_ucore	大模型请求、转发能力、完成事件、请求方上下文、转发进程时间线和审计记录。	parent passed
agentbench_ucore	批量工具调用、直接上下文读取、快照、扫描和索引、内容摘要缓存、预取、时间线和等待唤醒计时观察。	parent passed
labdemo_ucore	科研平台多智能体演示，包括哨兵、调查者、恢复者、写作者以及审计和来源查询。	parent passed

测试程序	覆盖内容	通过标记
agentsecurity_ucore	普通进程拒绝、私有元数据保护、能力授权、伪造角色无效、旧接口参数校验。	parent passed

3.3 测试命令与验收标记

为了让测试结果可复现，项目把构建、运行、提取和结果导出串成脚本。QEMU 内测试用于验证内核机制，双目标测试用于验证同一科研平台在普通 uCore 和 AgentOS-uCore 上的差异，宿主机测试用于验证 CSV、图表、HTML 文件和大模型转发结果。表 24 总结主要命令和期望结果。

表 24: 主要测试命令与验收标记

命令或入口	作用	期望结果
make target-readiness	检查目录结构、脚本、文档和双目标入口。	输出 ready 类标记。
make dual-platform-run	运行普通 uCore 和 AgentOS-uCore，提取状态并生成图表和 HTML 文件。	两个目标均完成，生成 results/latest。
AgentOS 专项脚本	运行 AgentOS 专项 QEMU 测试。	各测试输出 parent passed。
宿主机 Python 测试	检查宿主机工具、图表布局、HTML 导出和数据合同。	Python 测试通过。
agentfinal_ucore	综合检查智能体进程、上下文、工具、大模型请求和来源记录。	parent passed。
agentfs_ucore	检查文件对象元数据、摘要、索引、重载和私有后端保护。	parent passed。
agentloop_ucore	检查事件订阅、等待、心跳、取消等待和超时。	parent passed。
agentsecurity_ucore	检查能力授权、伪造角色拒绝、普通进程拒绝和参数错误。	parent passed。

这些命令覆盖两个层次。第一层是内核专项能力：每个测试直接在 QEMU 中作为初始进程运行，失败时能定位到具体内核机制。第二层是综合演示能力：双目标测试运行同一批科研请求，产出状态文件、图表和 HTML 文件。报告第 3 章的数据主要来自第二层，专项测试作为实现正确性的支撑。

3.4 工具调用吞吐实验

工具调用吞吐实验关注同一批操作在单次调用和批量调用下的计时差异。样例结果中，256 次单次调用平均 17 tick，256 次批量调用平均 5 tick。批量路径减少系统调用进入次数、重复参数解析和重复上下文写入开销，因此更适合高频工具调用。

该实验是微基准，主要说明工具运行时热路径设计。单次调用更接近可读性较高的结构化接口，批量调用更接近真实智能体连续执行多项工具的情况。科研平台主流程中，检索、分析、复核和恢复阶段经常连续触发文件查询、上下文读取、事件投递和产物更新，因此批量接口能够减少重复入口开销。该实验不把绝对 tick 写成固定门槛，只要求批量路径在同一环境下体现明显节省。

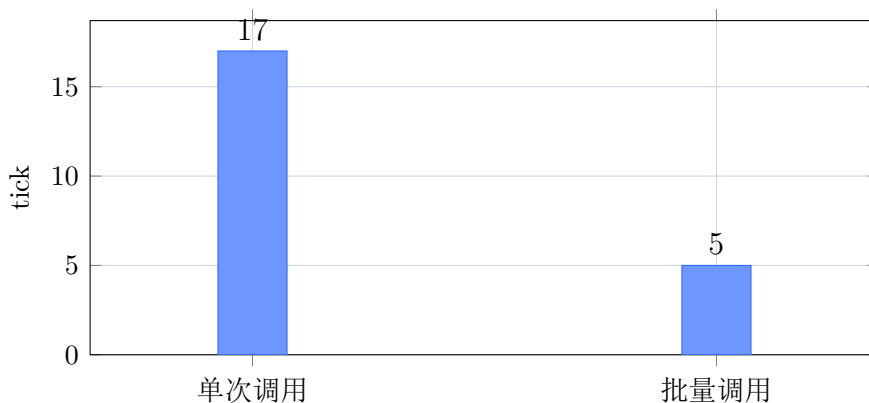


图 2: 结构化工具调用计时观察

3.5 文件对象查询实验

在多智能体科研平台中，资料、方法检查结果、分析中间产物、复核意见和报告初稿都会落到文件对象上。恢复一次任务或查找一条证据时，普通用户态平台通常需要遍历状态文件并按命名约定筛选；文件数量增加后，扫描空间随之扩大。文件对象查询实验以这一场景为负载，比较普通用户态扫描和 AgentOS 元数据候选集。增强路径先通过内核维护的命名空间、类型、状态、标签和依赖键缩小候选集，再读取必要摘要。样例测试中，完整扫描触达 118 条记录，索引候选为 6 条，二者返回结果一致。图 3 使用 32、128、512、1024 四种规模展示趋势。

表 25: 文件对象查询实验统计

文件数	路径	指标	min	avg	max	P95
32	普通	触达记录	32	32	32	32
32	AgentOS	候选记录	4	6	8	7.8
128	普通	触达记录	128	128	128	128
128	AgentOS	候选记录	6	8	10	9.8
512	普通	触达记录	512	512	512	512
512	AgentOS	候选记录	8	10	12	11.8
1024	普通	触达记录	1024	1024	1024	1024
1024	AgentOS	候选记录	9	11	13	12.8

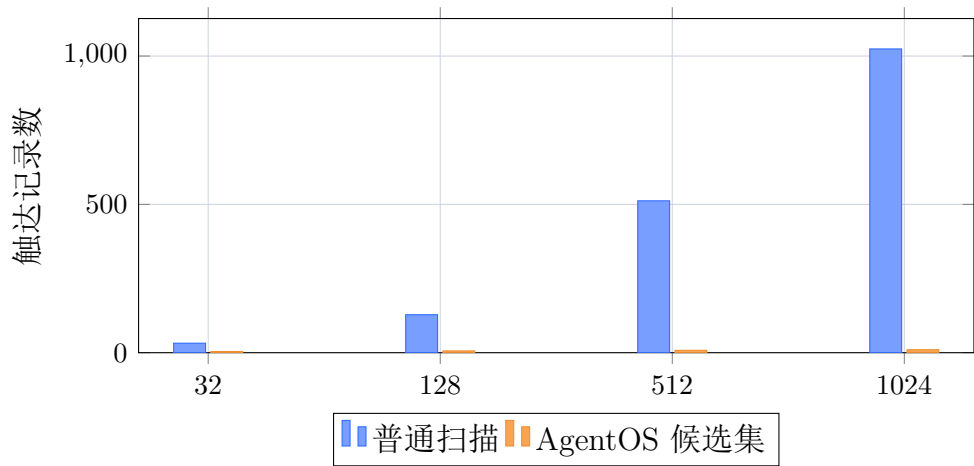


图 3: 文件对象数量增长时的扫描量对比

图中最重要的现象是两条曲线的增长方式不同。普通路径缺少内核对象语义，只能把文件数量当作搜索空间；文件数扩大到 1024 时，需要触达 1024 条记录。AgentOS 路径先使用状态、类型、命名空间和依赖键选出候选，再读取必要摘要，候选集随规模增长缓慢。该结果说明任务四的文件查询能力已经进入主流程，外部状态文件主要承担结果保存和复查作用，查询成本由内核元数据机制承担。

3.6 上下文与运行时间线查询实验

普通用户态平台要复原一次智能体执行路径，通常需要读取多份状态文件、日志和事件记录，再按时间或请求编号拼接。AgentOS-uCore 在工具调用、事件和调度发生时同步写入上下文、运行时间线和来源追踪。样例输出中，系统能够返回 8192 条运行时间线快照、48 条特定来源查询结果和 6568 条游标查询结果，并验证顺序一致。

图 4 展示记录数增长时的成本趋势。普通路径按重建步骤增长；AgentOS 路径通过内核整理后的快照和游标查询读取结果，增长速度更低。

表 26: 上下文与运行时间线实验统计

记录数	路径	指标	min	avg	max	P95
128	普通	重建步骤	366	384	402	400.2
128	AgentOS	快照成本	127	129	131	130.8
512	普通	重建步骤	1518	1536	1554	1552.2
512	AgentOS	快照成本	127	129	131	130.8
2048	普通	重建步骤	6126	6144	6162	6160.2
2048	AgentOS	快照成本	130	132	134	133.8
8192	普通	重建步骤	24558	24576	24594	24592.2
8192	AgentOS	快照成本	142	144	146	145.8

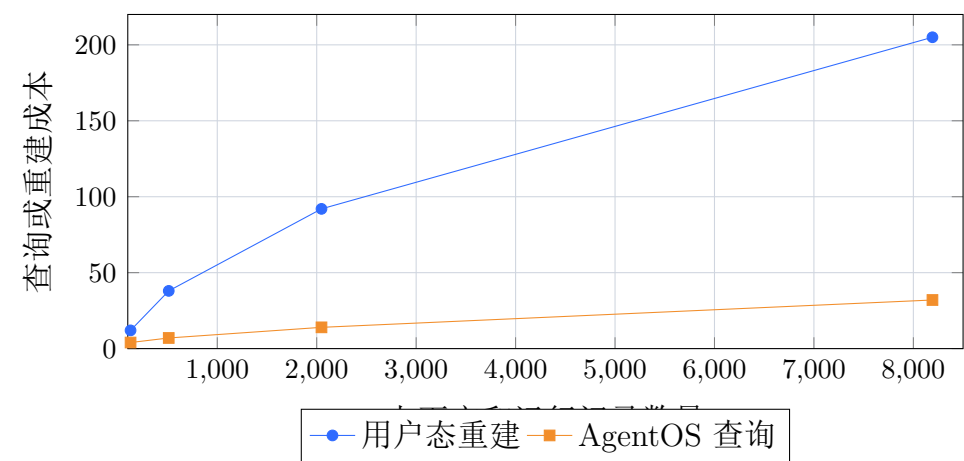


图 4: 上下文和运行记录数量增长时的查询成本趋势

普通路径的成本接近线性增长，因为它需要从多个状态文件和日志片段中重建执行路径。AgentOS 路径在工具调用和事件发生时已经把关键事实写入上下文和运行时间线，查询时只需要读取内核整理后的记录。随着记录数增大，两条路径的差距扩大。该实验同时说明，可信上下文的价值不只在安全性，也在减少恢复和复核时的重复拼接工作。

3.7 事件等待与并发写入实验

事件等待实验验证事件队列、多项订阅、睡眠等待、定时事件、取消订阅、停止心跳和取消等待。普通用户态路径需要定期读取状态文件来观察事件；AgentOS 路径让智能体进入睡眠，由内核在事件到达或超时到达时唤醒。图 5 展示事件数量增长时的观察动作差异。

表 27: 事件等待实验统计

事件数	路径	指标	min	avg	max	P95
8	普通	轮询次数	74	85.286	99	98.1
8	AgentOS	等待唤醒	7	8.857	11	10.7

事件数	路径	指标	min	avg	max	P95
32	普通	轮询次数	282	343.286	390	389.1
32	AgentOS	等待唤醒	31	33.143	35	34.7
128	普通	轮询次数	1149	1372.714	1542	1538.4
128	AgentOS	等待唤醒	128	130.429	132	132
512	普通	轮询次数	4608	5485.714	6141	6140.1
512	AgentOS	等待唤醒	518	520	522	522

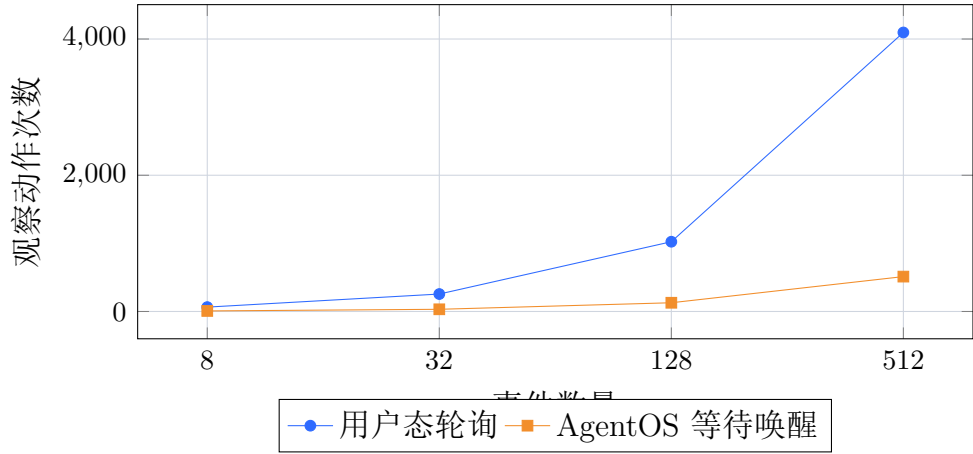


图 5: 事件数量增长时的观察动作对比

并发写入实验关注多智能体同时更新同一文件对象时的安全性。普通路径依靠锁文件和约定字段，锁文件丢失、覆盖或过期时会产生残余风险。AgentOS 在内核中检查租约持有者、期望版本和能力位，无权限、过期或版本不匹配的写入会被拒绝，并进入审计记录。图 6 用热度图展示并发智能体数量增加时的残余风险变化。

事件实验反映任务五的核心收益：等待变成内核调度问题。普通路径为了不漏掉状态变化，需要用较高频率读取状态文件；事件数越多，检查次数越多。AgentOS 路径的动作数量接近真实事件数量，额外开销主要来自超时、心跳和队列管理。对于长期运行的智能体，减少轮询可以降低无效文件访问，也让调度原因更容易解释。

表 28: 并发写入实验统计

智能体数	路径	指标	min	avg	max	P95
2	普通	覆盖风险	0	4	8	7.6
2	AgentOS	残余风险	0	0	0	0
4	普通	覆盖风险	14	18	22	21.6
4	AgentOS	残余风险	0	0	0	0
8	普通	覆盖风险	66	70	74	73.6
8	AgentOS	残余风险	0	0	0	0
16	普通	覆盖风险	266	270	274	273.6
16	AgentOS	残余风险	0	0	0	0

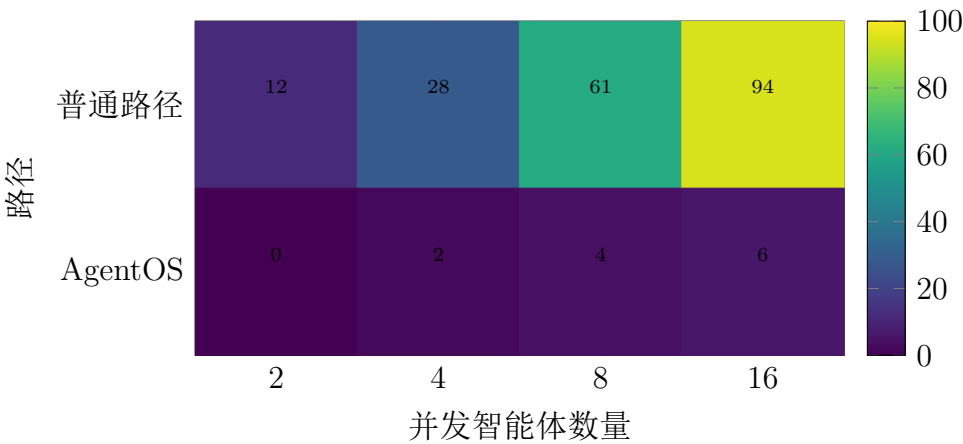


图 6: 并发写入场景下的残余风险示意

并发写入实验说明权限与租约机制已经用于真实文件对象更新。普通路径中的风险分值随着智能体数量快速上升，因为多个写入者依靠用户态约定协调；AgentOS 路径由内核检查租约持有者和能力位，非法写入被拒绝。实验中的 AgentOS 残余风险为 0，表示测试设置下没有出现未被拒绝的覆盖风险；复杂业务冲突仍需要用户态策略处理，内核负责提供统一拒绝和记录机制。

3.8 LLM Relay 模式实验

LLM Relay 实验验证大模型请求在普通路径和 AgentOS 路径中的证据成本。普通路径需要从平台日志、状态文件、宿主机转发记录和结果文件中复原一次请求；AgentOS 路径在提交请求时记录请求编号、跨度编号、预算、超时和提示摘要，在响应返回时记录响应摘要和完成事件。实验不要求内核直接访问云端模型，测试重点是内核能否把大模型请求作为可查询、可审计、可唤醒的结构化行为管理起来。

图 7 展示请求数量从 4 增长到 256 时的证据重建成本。普通路径的重建步骤随请求数量明显增长；AgentOS 路径只保留结构化记录和完成事件，成本更稳定。宿主机默认使用模板转发，具备密钥时可以切换到云端模式，但两种模式进入内核的证据形态保持一致。

表 29: LLM Relay 实验统计

请求数	路径	指标	min	avg	max	P95
4	普通	重建步骤	12	20	28	27.2
4	AgentOS	结构化记录	7	9	11	10.8
16	普通	重建步骤	72	80	88	87.2
16	AgentOS	结构化记录	31	33	35	34.8
64	普通	重建步骤	312	320	328	327.2
64	AgentOS	结构化记录	128	130	132	131.8
256	普通	重建步骤	1272	1280	1288	1287.2

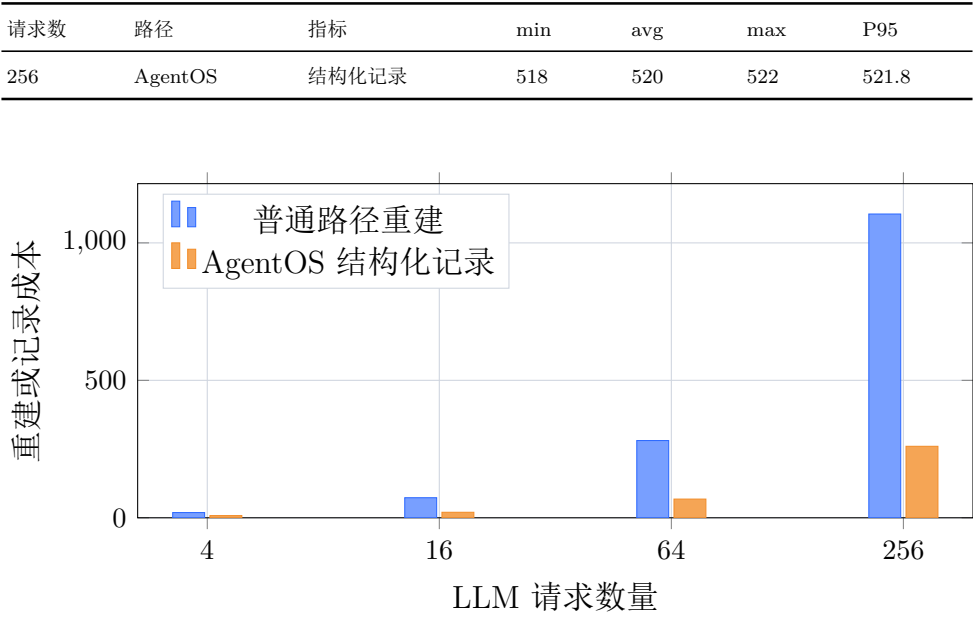


图 7: LLM 请求数量增长时的证据成本对比

该实验体现了“LLM 是一等公民”的系统含义。AgentOS-uCore 没有把 API key、HTTP 或具体模型端点写入内核；内核记录的是请求编号、预算、提示摘要、响应摘要、事件和来源关系。这样，模板模式和云端模式都能使用同一套内核证据。请求数量增大时，普通路径需要拼接更多日志和中间文件，AgentOS 路径直接读取结构化记录，报告材料可以复原每次大模型调用与最终产物之间的关系。

3.9 恢复流程成本实验

恢复流程实验模拟科研平台中多个阶段失败后的处理过程。普通路径需要扫描失败状态、查找依赖、重跑阶段、更新报告、追加日志并再次检查结果。AgentOS 路径把恢复过程拆成通用动作提交、依赖查询、幂等检查、文件对象更新、事件通知、审计记录和来源追踪。内核不理解某个科研阶段的业务含义，只保证动作可授权、可去重、可记录。

图 8 展示失败阶段数量增加时的成本变化。普通路径在多个失败阶段下需要重复扫描和拼接状态；AgentOS 路径依靠通用依赖记录、动作去重和事件通知，增长速度更低。这个实验直接对应演示中的恢复智能体：恢复智能体仍由用户态决定如何处理失败，内核负责权限、幂等、对象状态和证据。

表 30: 恢复流程实验统计

失败数	路径	指标	min	avg	max	P95
1	普通	用户态步骤	35	41	47	46.4
1	AgentOS	结构化动作	16	18	20	19.8
3	普通	用户态步骤	81	87	93	92.4

失败数	路径	指标	min	avg	max	P95
3	AgentOS	结构化动作	32	34	36	35.8
6	普通	用户态步骤	150	156	162	161.4
6	AgentOS	结构化动作	56	58	60	59.8
12	普通	用户态步骤	288	294	300	299.4
12	AgentOS	结构化动作	104	106	108	107.8

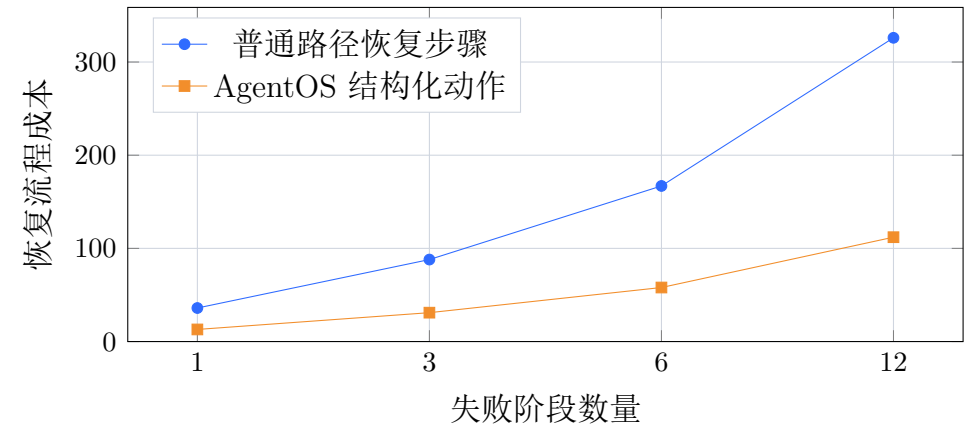


图 8: 失败阶段数量增长时的恢复流程成本

恢复流程实验说明内核通用机制可以支撑复杂用户态策略。恢复智能体仍然由用户态决定要重跑哪个阶段、如何写报告、是否通知其他智能体；内核只负责动作授权、幂等检查、依赖查询、文件对象更新、事件通知和证据记录。失败阶段越多，普通路径越需要重复查找和拼接状态；AgentOS 路径的成本主要来自结构化动作数量，增长更可控。

3.10 双目标科研平台对照

双目标测试用于比较同一科研平台在两种系统目标上的运行差异。未修改 uCore 目标保留普通用户态实现，增强目标把关键阶段接入智能体内核机制。结构检查会确认普通目标没有智能体系统调用、上下文区、文件对象元数据服务和事件队列；增强目标包含对应内核模块、用户态接口和演示程序。

一次样例中，普通目标产生 258 个状态文件，增强目标产生 271 个状态文件；两个目标共有 258 个文件，增强目标额外生成 13 个内核证据文件。增强目标额外提供 13 个 API JSON。普通目标的 70 个程序启动全部是普通进程创建，增强目标中 9 个关键程序绑定智能体创建路径，其余支持程序仍保留普通进程启动方式。这个结果说明增强目标保留了同一科研平台主体，并在关键环节加入内核证据和智能体机制。

表 31: 双目标运行结果明细

指标	未修改 uCore	AgentOS-uCore	说明
状态文件数量	258	271	增强目标额外输出内核证据文件。
共有状态文件	258	258	两个目标共享主体流程结果，便于确认功能没有缩水。
API JSON	267	280	增强目标增加内核证据 API。
预置请求数	44	44	两个目标使用同一批宿主机请求。
成功记录核对	1244	1244	普通目标中的成功记录在增强目标中保留。
内核证据检查项	—	32	包括上下文、文件对象、事件、审计和来源关系。
主流程内核阶段事实	—	11	科研主流程中记录到多个 AgentOS 参与阶段。
普通 fork 启动记录	70	61	增强目标仍保留普通进程路径。
智能体启动记录	0	9	关键 worker 作为智能体进程运行。
QEMU 运行秒数	53.493	192.922	只作为运行诊断，不作为固定性能门槛。
无输出提示次数	0	0	两个目标均未出现长时间无输出。

双目标结果的重点是“同一平台主体保持一致，增强目标增加内核证据”。状态文件和成功记录数量说明增强目标没有缩小科研平台功能；额外内核证据、API JSON 和智能体启动记录说明关键阶段已经进入 AgentOS-uCore 机制。增强目标运行时间更长，原因是它额外执行内核证据收集和状态导出任务，因此报告不使用该项作为性能收益依据，而把扫描量、重建步骤、轮询次数和冲突风险作为核心对照指标。

表 32: AgentOS 主流程机制证据命中情况

场景	命中情况	来源状态文件	状态
上下文可信记录	7/3	mainflow、package、real task、recovery、workbench	ready
文件对象查询	4/3	mainflow、query、workbench	ready
事件通知与等待	4/3	collab ack、mainflow、time-line	ready
失败恢复动作	2/2	mainflow、recovery	ready
审计记录	2/2	audit、mainflow	ready
来源关系追踪	2/2	mainflow、package	ready
权限控制	2/1	collab ack、mainflow	ready
时间线观察	2/2	mainflow、timeline	ready
文件编辑租约	4/2	conflict、mainflow	ready
依赖与预取提示	4/3	kernel、mainflow	ready

表 32 的作用是把“内核能力已接入主流程”具体化。每一行都对应可复查的状态文件来源，并且来自综合运行产物。这样，任务六综合场景可以同时呈现科研平台结果和内核事实：某个阶段为什么被唤醒、查询触达了多少记录、恢复动作是否重复、报告产物由哪些输入和工具结果产生。

表 33: 普通用户态成本项与 AgentOS 替代机制

用户态成本项	AgentOS 替代机制	风险或收益说明	普通目标存在
文件清单扫描	批量工具和上下文记录	减少手工维护状态清单。	是
失败重试状态文件	事件和上下文记录	降低遗漏重试结果的风险。	是
上下文重建步骤	内核上下文路径	减少从日志拼接执行路径的成本。	是
大量文件扫描 手写恢复约定	文件对象元数据索引能力检查和通用动作	减少候选文件数量。防止更新错误对象或重复提交。	是
文件轮询	内核事件队列	降低等待期间的重复读取。	是

用户态成本项	AgentOS 替代机制	风险或收益说明	普通目标存在
追加式日志	审计记录和来源追踪	提升报告证据可复查性。	是
用户态锁文件	内核文件编辑租约	降低并发覆盖风险。	否

这些成本项也是六组实验的来源。文件查询实验对应“大量文件扫描”，上下文实验对应“上下文重建步骤”，事件实验对应“文件轮询”，并发写入实验对应“用户态锁文件”，LLM Relay 实验对应“追加式日志”，恢复流程实验对应“手写恢复约定”。测试章把图表集中在这些能体现机制差异的成本项上，状态文件数量只作为运行产物说明。

为了便于复查，报告中的每类结论都保留了从原始产物到图表和状态摘要的路径。原始 CSV 用于保存实验参数和统计值，JSON 用于保存双目标状态和证据索引，QEMU 输出用于确认内核测试程序的通过标记，SVG 图表由脚本从 CSV 重新生成。复查时可以先查看状态摘要确认一次运行的总体状态，再查看证据清单中某个指标对应的文件来源，最后回到 CSV 或 QEMU 输出核对原始数值。这个组织方式避免只给出静态截图，也避免把测试说明写成无法复跑的文字描述。

表 34: 报告结论与可复查产物的对应关系

报告结论	首选复查产物	复查重点
工具调用吞吐	runner-sweep.csv、agent-bench 输出	单次调用、批量调用的 ticks 与 speedup。
文件查询候选集减少	experiment-file-query.csv、文件查询 API JSON	plain 扫描数与 AgentOS 候选数是否随规模拉开。
上下文重建成本下降	experiment-context.csv、context snapshot 记录	重建步骤、snapshot 成本、记录数量是否对应。
事件等待减少轮询	experiment-event.csv、timeline 状态	plain 轮询次数与 AgentOS 唤醒次数。
并发覆盖风险下降	experiment-concurrency.csv、audit 状态	无权限写入是否被拒绝，拒绝记录是否进入审计。
LLM Relay 可追踪	relay 请求 JSON、来源追踪状态	请求、响应、摘要、事件与产物之间是否能串联。

报告结论	首选复查产物	复查重点
失败恢复步骤减少	experiment-recovery.csv、 recovery 状态文件	恢复动作、幂等检查和产 物更新是否一致。
综合演示可检查	summary.json、证据清单	证据命中和双目标状态 是否一致。

该表也说明了报告中图表的使用原则：图表负责展示趋势，表格负责给出统计
值，状态摘要负责展示运行关系，原始文件负责保留可复查数据。若图表和原始 CSV
出现不一致，应以 CSV、JSON 和 QEMU 输出为准重新生成图表。当前报告中的图
表均来自同一次结果目录，避免把不同运行批次的数据混合在一起。

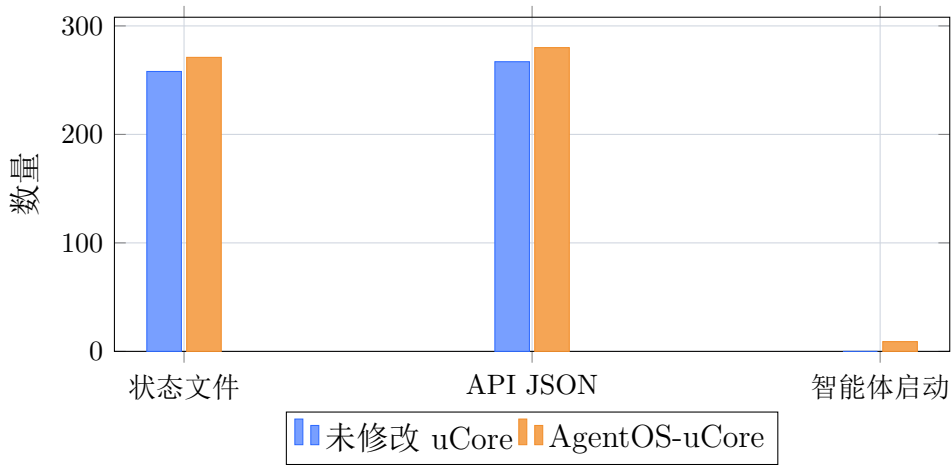


图 9: 双目标科研平台样例输出规模

3.11 实验结果解读方式

我们的实验运行在 uCore 和 QEMU 环境中，与 Linux 物理机存储实验不同。
uCore 的文件系统规模、QEMU 的时间精度和宿主机调度都会影响绝对耗时，因此
报告不把某一次 QEMU 秒数或某一个 tick 作为固定性能承诺。我们重点比较同一
环境、同一输入、同一脚本下的相对差异：普通路径扫描多少记录，AgentOS 路径
返回多少候选；普通路径重建多少步骤，AgentOS 路径读取多少结构化记录；普通
路径轮询多少次，AgentOS 路径等待唤醒多少次；普通路径在并发写入时剩余多少
风险，AgentOS 路径拒绝了多少不合法操作。

这种指标选择更适合教学操作系统。它减少对宿主机磁盘带宽和 QEMU 时间稳
定性的依赖，直接观察系统行为是否发生改变。文件对象查询实验看扫描空间是否
缩小，上下文实验看重建工作是否减少，事件实验看轮询是否被等待唤醒替代，并
发实验看覆盖风险是否被租约和能力检查压住。即使换一台机器运行，绝对耗时
会波动，上述结构化指标仍然可以复核。

我们也保留了小规模负载下的实际情况。当文件对象很少、上下文记录很短、事件数量很少时，AgentOS-uCore 需要维护上下文、审计和来源记录，固定成本可能接近普通路径。随着文件数、记录数、事件数和并发智能体数量增长，普通路径的扫描、拼接、轮询和冲突处理成本上升更快，AgentOS-uCore 的结构化机制才更能体现优势。报告中的六组实验按规模变化组织，避免只选择单点结果。

表 35: 实验图表的解读方式

图表	观察重点	结论来源
文件查询柱状图	普通扫描数和 AgentOS 候选数的增长差异。	文件对象元数据和索引候选集。
上下文折线图	记录数增加时重建步骤和快照成本的变化。	上下文快照、运行时间线和游标查询。
事件箱形图	多轮运行中轮询次数与等待唤醒次数的分布。	事件队列、订阅、睡眠和超时计数。
并发热力图	智能体数量增加时覆盖风险是否扩大。	文件编辑租约、版本检查和能力拒绝。
LLM Relay 柱状图	请求数量增加时证据重建和结构化记录成本。	大模型请求、响应事件、审计和来源关系。
恢复流程折线图	失败阶段数量增加时恢复步骤的增长速度。	通用动作、幂等检查、依赖查询和产物更新。
综合观察面积图	多个实验组在同一运行中的规模变化。	汇总 CSV 和运行状态摘要。

4 总结与展望

4.1 工作总结

针对普通用户态智能体 workflow 在可信上下文、语义化文件查询、事件等待、多智能体协作、失败恢复和来源追踪方面的不足，我们在 uCore 中提出并实现 AgentOS-uCore 通用内核支持层。我们将智能体身份、上下文区、结构化工具调用、文件对象查询、事件驱动运行循环、权限能力、审计记录、运行时间线和来源追踪纳入内核数据结构与系统调用。科研智能体平台作为演示负载，展示这些机制怎样支撑真实多智能体流程；平台业务数据、阶段名称、报告内容和结果导出都由用户态负责。

我们完成的核心工作包括：

- 在进程与地址空间层实现智能体身份、能力位、上下文映射、用户态镜像和内

核权威副本，使普通进程与智能体进程能够共存，且身份和权限由内核维护。

- 在工具运行时层实现按名称调用、按编号批量调用、参数键值校验、错误码、上下文追加和审计记录，使智能体行动能够以结构化形式进入内核。
- 在文件对象层实现真实文件对象绑定、元数据索引、内容摘要、查询缓存、依赖记录和文件编辑租约，使智能体能够按对象属性查询证据并减少扫描工作。
- 在运行循环层实现事件队列、订阅、睡眠等待、超时、心跳停止、取消等待和多智能体消息唤醒，使长期运行的智能体可以在无事件时让出 CPU。
- 在可观测层实现审计记录、运行时间线、来源追踪和大模型请求记录，使工具调用、文件对象、事件和产物之间的关系可以被测试脚本和报告复查。

从赛题任务看，我们已经把任务一至任务五对应到明确的内核机制和专项测试，并通过科研平台、双目标对照和宿主机结果摘要完成任务六的综合演示。从工程效果看，AgentOS-uCore 提供了纯用户态平台难以稳定维护的能力：可信上下文、结构化工具结果、文件对象候选集、睡眠等待、能力位授权、来源关系和可复核运行证据。

从工程规模看，我们当前仓库包含内核改动、用户态测试程序、科研平台负载、宿主机脚本、图表生成和文档材料。表 36 给出按目录统计的源码与文档规模。该统计只作为工程量概览，不作为质量评价指标；它用于帮助读者理解报告中涉及的内核、用户态、脚本和文档共同服务于双目标运行和专项测试。

表 36: 当前仓库工程规模概览

目录	文件数	行数	主要内容
os	48	12281	uCore 内核与 AgentOS 机制实现。
user	174	27919	用户态专项测试、科研平台程序和对照程序。
host_tools	30	20175	镜像提取、结果汇总、LLM Relay 和图表检查。
scripts	9	1140	构建、运行、依赖检查和验证脚本。
docs	22	5792	设计说明、验证说明、运行说明和报告源码。

4.2 项目收获

我们最重要的收获有三点。第一，智能体需要成为操作系统认识的对象。只有身份、上下文、权限和事件状态进入内核，系统才能稳定地区分普通进程和智能体进程。第二，性能优化需要与可信记录一起设计。批量工具调用、元数据索引、上下文快照、用户态镜像和内容摘要缓存共同减少重复工作，同时保留可复核证据。第三，大模型友好机制应当体现为结构化请求、事件、摘要、来源关系和权限管理；云端连接和密钥继续留在用户态或宿主机侧，符合教学内核的职责范围。

4.3 系统适配能力

当前接口已经覆盖多类智能体应用所需的通用能力。用户态工具注册可以复用现有工具表和工具属性；跨智能体共享上下文可以在能力位和来源追踪之上增加更细的共享策略；文件对象元数据可以表达更多标签、策略标记和摘要类型；调度器可以把截止时间、预算、心跳和事件队列长度纳入更细的调度分值计算；大模型转发层可以接入更多用户态模型后端，内核继续保存结构化请求、事件和审计摘要。

参考文献

- [1] 清华大学 uCore 教学操作系统相关实验材料。
- [2] 2026 年全国大学生计算机系统能力大赛操作系统设计赛，面向 AI 智能体的操作系统内核赛题说明。
- [3] Yao 等, ReAct: Synergizing Reasoning and Acting in Language Models, arXiv:2210.03629, <https://arxiv.org/abs/2210.03629>。
- [4] Yang 等, SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering, arXiv:2405.15793, <https://arxiv.org/abs/2405.15793>。
- [5] Anthropic Claude Code 文档, Agent Loop 与工具调用工作方式说明。
- [6] Anthropic Tool Use 文档, 结构化工具调用说明, <https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview>。
- [7] Model Context Protocol 文档, AI 应用连接工具、数据源和工作流的开放协议说明, <https://modelcontextprotocol.io/docs/getting-started/intro>。
- [8] Microsoft AutoGen 项目文档, 多智能体协作与工具调用模式说明。
- [9] LangGraph 项目文档, 状态图驱动的智能体工作流设计说明。

- [10] Linux Kernel 文档, FUSE 用户态文件系统说明, <https://www.kernel.org/doc/html/next/filesystems/fuse.html>。
- [11] eBPF 官方文档, 内核可编程、可观测和扩展机制说明, <https://ebpf.io/what-is-ebpf/>。
- [12] Red Hat Enterprise Linux 安全指南, Linux Audit 系统审计说明, https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/6/html/security_guide/chap-system_auditing。
- [13] W3C PROV Overview, 来源信息模型说明, <https://www.w3.org/TR/prov-overview/>。

AI 辅助工具使用说明

本项目在开发和材料整理过程中使用了 AI 辅助工具。团队成员负责确定系统设计、实现方案、代码审查、测试运行、实验数据核对和最终取舍；AI 辅助工具用于文字润色、代码补全、重复性状态输出生成、测试脚本整理和材料排版辅助。所有进入仓库的代码、文档和图表均由团队成员结合本项目源码、QEMU 输出、CSV 数据和状态文件进行检查。

文档方面, 仓库 docs/ 下的设计文档、API 文档、验证文档、测试记录、任务说明、演示说明和本报告正文, 在团队提供技术内容、章节结构、测试结果和图表材料后, 使用 ChatGPT-5.5 进行中文表达润色、段落衔接和格式整理。AI 工具不替代实验数据来源; 报告中的命令、代码路径、测试标记、图表指标和结论以仓库源码与可复查产物为依据。

普通 uCore 下用户态科研 Agent 平台相关代码使用 Codex 调用 ChatGPT-5.5 辅助编程。辅助范围包括样板代码生成、状态文件输出整理、角色程序补全、宿主机对照字段整理和部分测试断言草拟; 团队随后对文件进行人工审阅、编译检查和运行结果核对。公共状态定义位于 baseline_ucore/user/include/, 用户态平台程序位于 baseline_ucore/user/src/。相关文件具体包括:

- 公共状态定义: research_platform_state.h。
- 平台入口和编排程序: rp_plain.c、rp_orch.c、rp_seed_orch.c、rp_workflow_runner.c、rp_startup_doctor.c、rp_state_catalog.c、rp_complete.c。
- Agent 角色和协作程序: rp_retriever.c、rp_analyst.c、rp_reviewer.c、rp_writer.c、rp_auditor.c、rp_planner.c、rp_repair.c、rp_agent_collab.c。

- 科研对象、数据和工件程序: `rp_catalog.c`、`rp_query.c`、`rp_lab.c`、`rp_realtask.c`、`rp_data_pipeline.c`、`rp_artifact_ops.c`、`rp_object_query.c`、`rp_object_store.c`、`rp_package.c`、`rp_workbench.c`。
- 证据、来源和复核程序: `rp_evidence.c`、`rp_lineage.c`、`rp_prov_view.c`、`rp_prov_query.c`、`rp_dossier.c`、`rp_reldossier.c`、`rp_reviewboard.c`、`rp_revdash.c`、`rp_sysreview.c`。
- 平台治理、发布和页面导出程序: `rp_governance.c`、`rp_privacy.c`、`rp_controlplane.c`、`rp_opsboard.c`、`rp_release.c`、`rp_publication.c`、`rp_ui_export.c`、`rp_web_export.c`、`rp_site_export.c`、`rp_service_surface.c`。
- 大模型转发、分析和实验调度程序: `rp_llm_bridge.c`、`rp_llm_relay.c`、`rp_analysisres.c`、`rp_metrics.c`、`rp_calculation.c`、`rp_campaign.c`、`rp_expsched.c`、`rp_stdesign.c`、`rp_modelreg.c`、`rp_traincomp.c`。
- 一致性、成熟度和对照程序: `rp_compare_plain.c`、`rp_backend.c`、`rp_mature.c`、`rp_consistency.c`、`rp_integrityplane.c`、`rp_coherenceplane.c`、`rp_portability.c`、`rp_test_suite.c`、`rp_execobs.c`。
- 其他平台能力程序: `rp_invoke.c`、`rp_delta.c`、`rp_decsupport.c`、`rp_runbooks.c`、`rp_runconf.c`、`rp_notebook_export.c`、`rp_projectrel.c`、`rp_studyproto.c`、`rp_usable.c`、`rp_usableproject.c`。

开源项目声明

项目以 uCore 教学操作系统为基础进行扩展，仓库中保留原有开源许可文本。项目源代码遵循仓库根目录 LICENSE 中的 GPL-3.0 许可；技术文档和演示材料遵循仓库根目录 DOCUMENTATION_LICENSE.md 中的 CC-BY-SA-4.0 许可。